

Approximate Computing Survey, Part II: Application-Specific & Architectural Approximation Techniques and Applications

VASILEIOS LEON, National Technical University of Athens, Zografou, Greece
MUHAMMAD ABDULLAH HANIF, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates
GIORGOS ARMENIAKOS, National Technical University of Athens, Zografou, Greece
XUN JIAO, Villanova University, Villanova, United States
MUHAMMAD SHAFIQUE, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates
KIAMAL PEKMESTZI, National Technical University of Athens, Zografou, Greece
DIMITRIOS SOUDRIS, National Technical University of Athens, Zografou, Greece

The challenging deployment of compute-intensive applications from domains such as Artificial Intelligence (AI) and Digital Signal Processing (DSP), forces the community of computing systems to explore new design approaches. *Approximate Computing* appears as an emerging solution, allowing to tune the quality of results in the design of a system in order to improve the energy efficiency and/or performance. This radical paradigm shift has attracted interest from both academia and industry, resulting in significant research on approximation techniques and methodologies at different design layers (from system down to integrated circuits). Motivated by the wide appeal of Approximate Computing over the last 10 years, we conduct a two-part survey to cover key aspects (e.g., terminology and applications) and review the state-of-the-art approximation techniques from all layers of the traditional computing stack. Part II of the survey classifies and presents the technical details of application-specific and architectural approximation techniques, which both target the design of resource-efficient processors/accelerators and systems. Moreover, it reports a quantitative analysis of the techniques and a detailed analysis of the application spectrum of Approximate Computing, and finally, it discusses open challenges and future directions.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software notations and tools**; • **Hardware** → **Integrated circuits**; • **Computer systems organization** → **Architectures;**

This research is partially supported by ASPIRE, the technology program management pillar of Abu Dhabi's Advanced Technology Research Council (ATRC), via the ASPIRE Awards for Research Excellence.
Authors' Contact Information: Vasileios Leon, National Technical University of Athens, Zografou, Greece; e-mail: vleon@microlab.ntua.gr; Muhammad Abdullah Hanif, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates; e-mail: mh6117@nyu.edu; Giorgos Armeniakos, National Technical University of Athens, Zografou, Greece; e-mail: armeniakos@microlab.ntua.gr; Xun Jiao, Villanova University, Villanova, Pennsylvania, United States; e-mail: xun.jiao@villanova.edu; Muhammad Shafique, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates; e-mail: muhammad.shafique@nyu.edu; Kiamal Pekmestzi, National Technical University of Athens, Zografou, Greece; e-mail: pekmes@microlab.ntua.gr; Dimitrios Soudris, National Technical University of Athens, Zografou, Greece; e-mail: dsoudris@microlab.ntua.gr.



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0360-0300/2025/02-ART177
https://doi.org/10.1145/3711683

Additional Key Words and Phrases: Inexact computing, approximation method, approximate programming, approximation framework, approximate processor, approximate memory, approximate circuit, error resilience, accuracy, digital signal processing, artificial neural networks, machine learning

ACM Reference Format:
Vasileios Leon, Muhammad Abdullah Hanif, Giorgos Armeniakos, Xun Jiao, Muhammad Shafique, Kiamal Pekmestzi, and Dimitrios Soudris. 2025. Approximate Computing Survey, Part II: Application-Specific & Architectural Approximation Techniques and Applications. *ACM Comput. Surv.* 57, 7, Article 177 (February 2025), 36 pages. https://doi.org/10.1145/3711683

1 Introduction

The recent technological advancements in processing, storage, communication, and sensing have transformed the landscape of computing systems. With the emergence of **Internet of Things** (**IoT**), a huge amount of data is generated, imposing technical challenges in the resource-constrained embedded systems that are placed at the edge of the network. The ever-growing number of global IoT connections, which increased by 18% in 2022 and is expected to be 29.7 billion in 2027 [94], only deteriorates this situation. At the same time, the already stressed cloud data centers are drawing worldwide concerns due to their enormous electricity demands. Indicatively, it is estimated that the energy consumption of data centers in Europe was increased by 43% from 2010 to 2020 [23], while according to Ref. [31], data centers may generate up to 8% of the global carbon emissions in 2030.

The above issues become even more critical when considering the massive growth of demanding applications from domains such as **Digital Signal Processing** (**DSP**), **Artificial Intelligence** (**AI**), and **Machine Learning** (**ML**). More specifically, the emergence of compute- and memory-intensive DSP and AI/ML applications marks a new era for computing, where traditional design approaches and conventional processors cannot meet the performance requirements [66, 169]. Therefore, it is obvious that the proliferation of data and workloads, the increase in the complexity of applications, and the urgent need for power efficiency and fast processing, all together, force the industry of computing systems to examine alternative design solutions and computing schemes.

In this transition, ***Approximate Computing (AxC)*** is already considered as a novel design paradigm that improves the energy efficiency and performance of computing systems [265]. During the past decade, AxC has been established as a vigorous approach to leverage the inherent ability of numerous applications to produce results of *acceptable* quality, despite some *inaccuracies* in the computations [17]. The motivation behind AxC is further analyzed in Part I of the survey [132]. Driven by its potential to address the energy and performance demands in a favorable manner, AxC is increasingly gaining significant attention over the last years. A representative example of this trend is depicted in Figure 1. This figure illustrates the growing number of works that apply any kind of approximation (e.g., software, hardware, architectural), published in four major design automation conferences. Furthermore, AxC has attracted significant interest from the industry. Companies such as Google [104], IBM [6], and Samsung [191] exploit the AI/ML error resilience and design accelerators for compressed networks with reduced precision and less computations, improving the energy efficiency and performance in exchange for negligible or zero accuracy loss.

Considering the need to go beyond single optimizations or efficient processor design, which solely do not suffice [50], prior research has focused on approximation techniques at different design abstraction layers [265], involving algorithmic- or compiler-level approximations (*software*) [9, 10], circuit-level approximations (*hardware*) [37, 99] and systematic design of approximate processors (*architectural*) [48, 57]. Despite their shared objective of achieving energy/area gains and latency reduction, these fundamental techniques are orthogonal and can complement

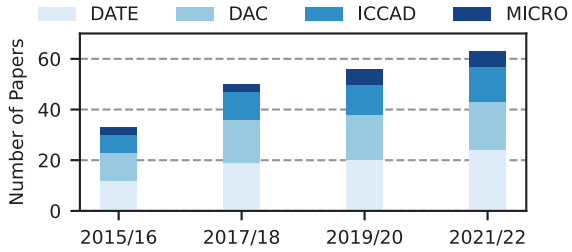


Fig. 1. Number of publications that apply any type of approximation in four major design automation conferences.

Table 1. Qualitative Comparison of Approximate Computing Surveys on the Entire Computing Stack

AxC Survey	Year Coverage	Pages #	References #	SW Tech.	HW Tech.	Arch. Approx.	AI/ML	Memories	Frameworks and Tools	Metrics	Benchmarks	CPU/FPGA/GPU/ASIC	Terminology	Challenges	Quant. Analysis[2]
[76]	2013	6	65	✓	✓	≈	≈	✗	✗	✓	✗	≈	✗	✗	≈
[168]	2015	33[1]	84	✓	≈	✓	≈	✓	✓	✓	✓	✓	✗	✓	✗
[281]	2015	15	59	✓	✓	✓	✗	≈	✗	✗	✗	≈	✗	✗	✗
[267]	2015	6	54	✓	✓	✓	✗	✗	✓	✗	✗	≈	✗	✗	✗
[232]	2016	6	47	✓	✓	✓	✗	✗	≈	✗	✗	≈	✗	✗	≈
[172]	2017	4	40	✓	✓	✗	≈	≈	✗	✗	✗	≈	✗	✗	✗
[15]	2017	6	72	✓	✓	✓	≈	≈	≈	✗	✓	≈	✗	✓	✗
[243]	2020	39[1]	235	≈	✓	✓	✗	✓	≈	✗	✗	≈	✗	✓	✗
This work Pt. 1	2023	34[1]	222	✓	✓				✓			✓	✓		✓
This work Pt. 2	2023	35[1]	301	✓	✓	✓	✓	✓		✓	✓	✓		✓	✓

[1]Single-column pages.
[2]Quantitative analysis involving count of works, frequencies and numerical assessments.

each other. As combining AxC techniques is expected to be highly beneficial (several works report energy gains over 90% [50]), a further adoption and understanding of cross-layer development (i.e., hardware/software co-design), including tools and frameworks tailored to specific applications, is highly required to maximize these benefits. In this context, motivated by the attractive outcomes, novel methods, and future potential of AxC, we conduct a two-part survey that covers all its key aspects.

2 Approximate Computing Survey

Scope and Contribution: As also discussed in Part I [132], the proposed survey examines the entire AxC computing stack, reviewing software, hardware, and architectural approximation techniques, contrary to other AxC surveys that focus on a single area (either design layer, technique or application). Table 1 compares surveys studying multiple areas of AxC, like our work. As shown, the proposed two-part survey includes all the areas examined in the previous works, while it reviews all of them in depth (e.g., AI/ML has not received significant attention) and it is up-to-date.

Regarding the contribution of the current survey, it provides the reader with a complete view on AxC, while it allows them to emphasize on specific areas (e.g., architectural techniques or application domains), as they are all presented in detail. Furthermore, this survey can be considered

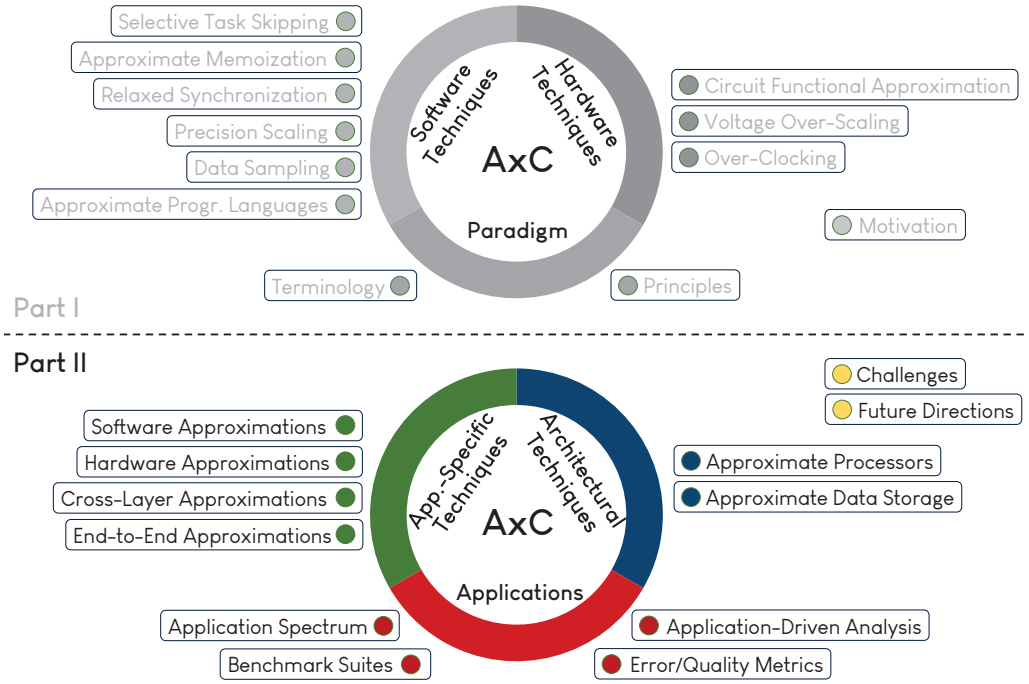


Fig. 2. Organization of the proposed two-part survey on Approximate Computing.

as a tutorial on the state-of-the-art approximation techniques, as it not only classifies them, but it reports in brief their technical details. Overall, the key contributions of the proposed two-part survey are: (1) the introduction of the AxC paradigm by discussing its background and application domains, (2) the review of a plethora of state-of-the-art approximation techniques, their coarse- and fine-grained classification, and the presentation of their technical details, (3) the study of application domains where approximations are tolerated and the analysis of representative techniques and results per domain, and (4) the discussion of open challenges in the design of approximate systems, as they emerge from the comprehensive literature review.

Organization: Figure 2 presents the content of each survey part:

Part I: It is presented in Ref. [132], and it introduces the AxC paradigm (terminology and principles) and reviews software and hardware approximation techniques.

Part II: It is presented in the current paper, and it reviews application-specific and architectural approximation techniques and introduces the AxC applications (domains, quality metrics, benchmarks).

The remainder of the article (Part II of the survey) is organized as follows. Section 3 reports software-, hardware- and cross-layer application-specific approximation techniques. Section 4 focuses on the architecture level and classifies approximate processors and storage. Section 5 performs a quantitative analysis of the reviewed works. Section 6 studies the applications of AxC, while Section 7 discusses challenges and future directions. Finally, Section 8 draws the conclusions.

3 Application-Specific Approximation Techniques

This section reviews application-specific approximation techniques. It mainly discusses techniques that are proposed for improving the performance and efficiency of **Deep Learning** (**DL**) and AI

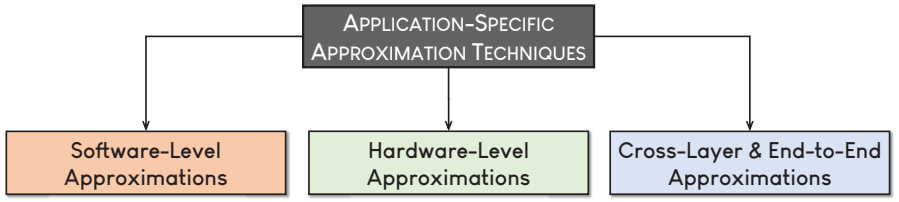


Fig. 3. Classification of application-specific approximation techniques for DL and AI applications in three main classes: *Software-Level*, *Hardware-Level*, and *Cross-Layer and End-to-End*.

Table 2. Classification of Application-Specific Approximation Techniques

Application-Specific Approximation Class	Technique/Approach
Software-Level Approximation	Computation Skipping / Pruning [43, 78, 79, 87, 88, 91, 128, 139, 246, 249, 272, 278, 288–290]
	Precision Scaling [78, 92, 97, 211, 274, 300]
	Input-Adaptive Approximation [65, 151, 184, 186, 195, 251, 266, 268]
	Input Scaling [67, 250]
Hardware-Level Approximation	Functional Approx. and Approx. Datapaths [80, 113, 131, 133, 175–177, 218, 238, 241, 269]
	Approximation-based Novel Data Representation Formats [72, 81, 96, 97]
	Voltage Scaling [98, 114, 118, 187, 236, 297]
	Data Approximation [114, 118]
Cross-Layer and End-to-End Approximation	Approx. at Various Layers of Computing Stack [19, 20, 59, 67, 68, 71, 82, 85, 202, 203, 265]
	Full-System Perspective [52, 67, 68, 85, 202, 203]
	Approx. of Multiple Subsystems in a Synergistic Manner [52, 67, 68, 85, 202, 203]

applications. The techniques presented in the literature can be broadly categorized into software-level, hardware-level and cross-layer approximation techniques (see Figure 3 and Table 2).

3.1 Software-Level Approximations

Various types of software-level approximation techniques have been proposed in the literature to reduce the computational complexity and memory footprint of AI and DL workloads. These techniques can be broadly divided into four groups: (i) *computation skipping / pruning* [43, 78, 79, 87, 88, 91, 128, 139, 246, 249, 272, 278, 288–290], (ii) *precision scaling* [78, 92, 97, 211, 274, 300], (iii) *input-adaptive approximation* [65, 123, 151, 184, 186, 195, 251, 266, 268], and (iv) *input scaling* [67, 250].

3.1.1 Pruning. Computation skipping is one of the most effective software-level techniques for approximating DNNs. Various techniques have been proposed under the umbrella of DNN pruning that exploits computation skipping for optimizing the DNN inference process. In general, DNN pruning refers to the process of removing non-essential (or less important) weights from a DNN to reduce its computational complexity, memory footprint as well as energy consumption during the inference process. These techniques are generally divided into two main categories: (1) unstructured pruning and (2) structured pruning.

Unstructured pruning (a.k.a. fine-grained pruning) refers to techniques that analyze the importance/saliency of each individual weight and remove it if its importance is below a predefined threshold. These techniques are usually designed to be iterative, where in each iteration only a small percentage of less significant weights are removed and then the network is retrained for a few epochs to regain close to the baseline accuracy. For example, Han et al. [79] propose a three-step method to prune DNNs. First, they train a DNN to learn the importance of connections. Then, they remove non-essential (less important) connections from the DNN. Finally, they fine-tune the remaining weights to regain the lost accuracy. The authors further propose that learning the

right connections is an iterative process. Therefore, the process of pruning followed by fine tuning should be repeated multiple times (with a small pruning ratio) to achieve higher compression ratios. Deep Compression [78] combines a similar method with weight sharing and Huffman coding to achieve ultra-high compression ratios.

Although unstructured pruning significantly reduces the number of parameters and the memory footprint of DNNs, it does not guarantee energy or latency benefits in all cases. This is mainly because; (1) the weights of a pruned network are usually stored in a compressed format, such as the **Compressed Sparse Row** (**CSR**) format, to achieve high compression ratios and, therefore, have to be uncompressed before corresponding computations, which results in significant overheads, and (2) the underlying hardware (in most use cases) is not designed to take advantage of unstructured sparsity in DNNs, which results in under utilization of the hardware. Therefore, specialized hardware accelerators, such as EIE [77] and SCNN [188], are required to efficiently process data using unstructured sparse DNNs. Similarly, Yu et al. [288] highlight that it is essential to align the pruning method to the underlying hardware architecture to achieve efficiency gains. A mismatch between the type of sparsity in the network and the sparsity the hardware can support efficiently usually leads to significant overheads. Based on this observation, the authors propose two novel pruning methods, i.e., SIMD-aware weight pruning and node pruning. SIMD-aware weight pruning maintains weights in aligned fixed-size groups to fully utilize the available SIMD units in the underlying hardware, while node pruning removes less significant nodes from the network without affecting the dense format of weight matrices.

Various other techniques have also been proposed in the literature to achieve structured sparsity in DNNs. These methods generally include weight-dependent pruning techniques [139, 289, 290], activation-based techniques [87, 88, 91, 246, 249], and regularization-based techniques [43, 128, 272, 278]. The weight-dependent techniques evaluate the importance of filters solely based on the weights of the filters, while activation-based techniques use activation maps to identify critical filters in the network. Similarly, regularization-based techniques encourage sparsity during training by adding regularization terms in the loss function or by exploiting batch normalization parameters.

3.1.2 Precision Scaling. It refers to the techniques of mapping values from a high-precision range to a lower-precision range with a lesser number of quantization level. The number of quantization levels in a range defines the number of bits required to represent each data value. Hence, by using a lower-precision format, such as 4-bit or 8-bit fixed-point or floating-point format instead of the standard 32-bit floating-point format, the memory footprint as well as the computational requirements of DNNs can be significantly reduced.

Various techniques have been proposed to achieve precision scaling of DNNs without affecting their application-level accuracy. The most commonly used technique is 8-bit range linear post-training quantization, where floating-point weights and activations are converted to 8-bit fixed-point format. The range linear quantization is further divided into two types: (1) asymmetric quantization [95] and (2) symmetric quantization [119]. In asymmetric quantization, the minimum and maximum observed values in the input range are mapped to the minimum and maximum values in the output (integer/fixed-point) range, respectively. However, in symmetric quantization, the maximum absolute observed value in the input range is used to define both minimum and maximum values of the input range. Significant efforts have been made to push the limits of DNN precision scaling to ultra-low precision levels. Works like XNOR-Net [211], **Binarized Neural Network** (**BNN**) [92] and DoReFa-Net [300] explore the potential of aggressive quantization, up to 1-bit precision. However, as aggressive quantization leads to significant accuracy loss, these techniques usually employ quantization-aware

training to achieve reasonable accuracy, as in such cases training also acts as an error-healing process.

Non-linear (or non-uniform) quantization techniques have also been explored in the literature [78, 97]. These techniques are inspired by the non-uniform probability distribution of DNN data structures and propose to distribute the quantization levels accordingly. Apart from the above-mentioned techniques, mixed-precision techniques are also commonly used, where different precision formats can be used for different layers and filters/channels of a DNN [274]. It is noted that mixed-precision quantization is only effective when the underlying hardware is capable of supporting different precision formats.

3.1.3 Input-Adaptive Approximation. Besides pruning and precision scaling, input-adaptive approximations are also commonly used to approximate DNN workloads. In general, input-adaptive approximations refer to the techniques that dynamically adapt the level of approximation based on the characteristics of the input data. Various methods have been proposed in the literature to realize input-adaptive approximations. These methods mainly include (1) early-exit classifiers, (2) dynamic network architectures, and (3) selective attention mechanisms. Works such as [184, 268] that fall under early-exit classifiers argue that most of the inputs can be classified with very low effort. Toward this, the authors in Ref. [184] propose **Conditional Deep Learning** (**CDL**), where the outputs of convolutional layers are used to estimate the difficulty of input samples (with the help of linear classifiers) and conditionally activate the deeper layers of the DNN. Similarly, Ref. [186] proposes a hierarchical classification framework by exploiting semantic decomposition. Toward dynamic network architectures, Ref. [65] presents an approach that adapts the structure of a DNN based on the difficulty of the input sample. Along similar lines, Ref. [251] presents a method for building runtime configurable DNNs and a score margin-based mechanism for dynamically adjusting the DNN capability. Unlike the above techniques that involve network structure or size modification, selective attention mechanisms focus on reducing computational requirements by processing only the important/relevant regions of the input while ignoring all less relevant parts. A prominent work in this direction includes selective tile processing [195], where a low cost attention mechanism is used to identify the tiles that require processing.

3.1.4 Input Scaling. Given that neighboring pixels in images are not always statistically independent and contain a significant amount of redundant information, input scaling can be exploited to reduce the computational requirements of DNN inference. Toward this, Tan et al. in Ref. [250] propose a compound scaling method that uniformly scales network width, depth, and input resolution to design a complete class of EfficientNets. Moreover, techniques such as AxIS [67] employ input scaling as an approximation knob in their cross-layer approximations to achieve ultra-high efficiency gains.

3.2 Hardware-Level Approximations

Venkataramani et al. in Ref. [269] highlight that DNNs are mostly used for error-resilient applications, and hence, hardware approximations can be used to effectively trade quality for efficiency of the DNN inference process. Toward this, they propose a method to transform a given **Neural Network** (**NN**) to an **Approximate Neural Network** (**AxNN**) [269]. The method mainly employs backpropagation to estimate the importance of each neuron and then selectively approximates less significant neurons through precision scaling of weights and activations to achieve better energy-accuracy tradeoffs. The method also employs approximation-aware retraining to regain a significant portion of the accuracy lost due to approximations.

As a single DNN inference can take from some million to billions of MAC operations, the use of approximate MAC units in DNN accelerators (instead of accurate ones) can have a significant

impact on the overall energy consumption of DNN inference systems. In general, approximate MAC units refer to the simplified variants of the accurate counterparts that offer approximately the same functionality but at a lower energy or performance cost. Various significant efforts have been made toward designing approximate MAC units. Most of these works focus on approximating the multiplication part in the MAC operations, as multipliers are in general more complex and resource hungry than adders. Marazek et al. in Ref. [175] employ the **Cartesian Genetic Programming** (**CGP**) approach to approximate accurate multiplier designs. The methodology employs approximation-aware retraining of the given NN in the methodology to reach the best possible approximate multiplier design for the given application. However, approximation-aware retraining is not possible in all scenarios, e.g., due to limited computational resources or due to the lack of availability of training data. Therefore, to address this limitation, ALWANN [177] proposes a layer-wise approximation technique to select an appropriate approximate multiplier from a given library (e.g., EvoApprox [174]) for each individual layer of the given DNN without involving any training. The authors also present a computationally inexpensive method to fine-tune the DNN weights after the approximate module selection process to reduce the accuracy loss due to approximations. In the same context, the Max-DNN framework [133] relies on ALWANN's functionalities to provide fine-grained approximation of the multiplications at different DNN layers. In Ref. [176], the authors present a methodology to generate libraries of more specific approximate circuits and show that such techniques can produce better results than techniques such as ALWANN [177]. Other prominent approximate multiplier designs for DNN acceleration include **Digital Approximate In-SRAM Multiplier** (**DAISM**) [238], positive/negative approximate multipliers [241] and approximate log-multipliers [113, 218]. Moreover, toward mitigating the impact of approximation errors, CANN [80] presents the concept of *curable* approximations for DNN accelerators, where approximation errors can be internally compensated with some additional functionality. To facilitate the design and exploration of approximate modules for efficient DNN inference, fast simulation techniques and frameworks are also being developed and studied [54, 70, 247]. Finally, the use of approximate multipliers provides resource saving and increased throughput in FPGA-based DNN accelerators [131], while the same benefits are also delivered in classic DSP kernels [135].

Another set of techniques that includes works such as BiScaled-DNNs [97], CoNLoCNN [81], Compensated-DNNs [96] and ANT [72], targets novel data representation formats to significantly reduce the hardware complexity and energy requirements of DNN inference systems. BiScaled-DNNs [97] argues that DNN data structures are long-tailed and, therefore, they can be effectively quantized using two different scale factors (namely, scale-fine and scale-wide). Scale-fine enables the system to precisely capture small numbers using more fractional bits, while scale-wide allows the system to cover the entire range of large numbers but at a coarser granularity. CoNLoCNN [81] employs a modified version of power-of-two quantization to replace multiplication with shift operations. Compensated-DNNs [96] propose the concept of dynamic compensation to mitigate the impact of quantization errors caused by aggressive quantization. On the other hand, ANT [72] argues that most of the existing quantization techniques use fixed-point or floating-point data representation formats, which offer limited benefits, as both are effective for specific data distributions, and thus, require a reasonably large number of bits to maintain the DNN accuracy. Therefore, [72] proposes a novel adaptive numerical data type that combines the benefits of float and integer data formats and, thereby, it allows the system to adapt based on the importance of each value in a tensor.

Apart from functional approximation of arithmetic modules in DNN accelerators, voltage-scaling can also be employed to improve the energy efficiency of DNNs. ThunderVolt [297] presents TE-Drop, a timing error recovery technique for DNN accelerators. It employs Razor flip-flops to detect timing errors in the MAC units and then avoids re-execution penalty by dropping the

MAC operation subsequent to the erroneous MAC. Ref. [236] presents a similar approach but employs Razor flip-flops in between the multiplier and the adder of each MAC unit. As dropping MAC operations can result in significant performance degradation, the authors in Ref.[98] present a **Compensation MAC** (**CMAC**) unit that compensates for the dropped multiplication using an additional adder in the MAC subsequent to the erroneous MAC. Along similar lines, GreenTPU [187] presents a **Timing Error Control Unit** (**TECU**) to keep track of the timing error causing input patterns and boost the operating voltage of the subsequent MACs in the processing array. Apart from exploiting voltage scaling for improving energy efficiency of computational units, works such as MATIC [114] explore the potential of aggressive voltage scaling of on-chip weight memories to improve the energy efficiency of DNN accelerators. Similarly, EDEN [118] presents a framework for using approximate DRAM with reduced voltage and reduced latency to improve the energy and performance efficiency of DNN inference systems. These techniques mainly employ memory adaptive training to achieve ultra-high efficiency gains and without any significant loss in the DNN accuracy.

3.3 Cross-Layer and End-to-End Approximations

The adoption of approximations in real-world systems has been mostly limited to either the software or hardware level and to only one subsystem (usually the computational one). To uncover the true potential of Approximate Computing, a full-system approach is essential where approximations are applied at multiple abstraction levels and to multiple subsystems (e.g., the memory, sensor, and communication subsystems) together with the computational subsystem. Various studies have shown the potential of systematic joint approximation of subsystems to achieve ultra-high efficiency gains. Earlier works highlight the potential of combining software- and hardware-level approximations. For example, in Ref. [185], the authors highlight different algorithmic and circuit-level approximation techniques for significantly reducing the energy consumption of DNN inference systems. For algorithmic approximations, they mainly discuss scalable effort computing techniques [184, 268], in which less complex inputs are processed using smaller networks, and more complex inputs are passed through a larger number of processing layers to generate an accurate output. For circuit/hardware-level approximations, they mainly highlight voltage scaling of on-chip SRAM and functional approximation of multipliers to reduce the energy consumption of DNNs.

In Ref. [202], Raha et al. present a full-system perspective for approximating smart camera systems designed for computer vision and image processing applications. They present a systematic methodology for judiciously approximating sensor, memory, and computation subsystems. For sensor approximation, they consider pixel subsampling (or downscaling image resolution), as it not only reduces the energy consumption of the sensor, but it also reduces the entire system's energy consumption by decreasing the input size (or data) and the number of computations required for processing the input. For approximating memory, they focus on increasing the refresh interval of DRAM, as sub-optimal refresh intervals result in a significant reduction in refresh energy consumption while still offering space to store critical data in non-erroneous DRAM pages. For computation approximation, they consider loop perforation and early loop termination techniques. To systematically tune all the subsystem approximation knobs, they first find the optimal image resolution for the given quality constraint and then using gradient descent over the individual quality vs. energy curves find the optimal operating points for DRAM and computation skipping. In Ref. [203], the methodology is extended to cover the communication subsystem as well. The work shows that for compute-intensive systems the methodology offers 3.5×–5.5× energy benefits, while for communication-intensive systems, e.g., where the input is sent to a server for processing, the methodology offers 1.8×–3.7× energy benefits at the cost of less than 1% application-level

quality loss. In Ref. [67], Ghosh et al. design a similar methodology for DNN-based inference system and demonstrate that their methodology can lead to 1.6×–1.7× energy benefits for the on-device inference scenario and 1.4×–3.4× energy benefits for the on-cloud inference scenario at the cost of less than 1% accuracy loss. The extension of this work is presented in Ref. [68], where various approximation techniques are applied: (i) compute approximation via a quality-driven model compression framework, (ii) memory approximation by reducing the DRAM refresh rate and allocating the weights and data in quality bins, (iii) sensor approximation using image subsampling in the camera, and (iv) communication approximation using lossy network data compression. Finally, an end-to-end approach for approximating an iris scanning system is presented in Ref. [85]. The authors employ approximations both at the software and hardware levels to build an accurate system using approximate pipeline. They identify eight different approximation knobs and then employ a reinforcement learning-based methodology for design space exploration on the approximate pipeline to achieve optimal tradeoff between accuracy and runtime/energy consumption. Their method can lead to 48× improvement in runtime while maintaining 100% accuracy.

The concept of distributed approximate systems is presented in Ref. [52], where a multi-device approach is adopted using synergistic approximations of edge computing systems to enable energy-efficient collaborative inference of distributed DNNs. This work performs significance-aware approximation schemes to prune the nodes and reduce the huge design space, while it also applies a heuristic to dynamically determine the approximation knobs for all the subsystems of all nodes under a given quality specification.

Apart from the above methods, various other techniques have also been proposed that combine DNN compression/approximation algorithms with hardware-level approximations to achieve significant efficiency gains. Gong et al. [71] present a framework that combines dynamic layered CNN structure, kernel shrinking and layer-by-layer quantization at the algorithm level and uses an approximate computing-based reconfigurable architecture at the hardware level. Similarly, Ref. [82] presents a cross-layer framework that combines network pruning and quantization with hardware-level functional approximations of arithmetic units. Along similar lines, Ref. [59] integrates activation pruning and voltage scaling with hardware-level functional approximation. To overcome the accuracy loss incurred due to hardware approximations, the authors propose to incorporate approximations in the training process, which results in an increase of about 5.32% in accuracy. The authors report around 52.5% improvement in the energy efficiency of the system on average.

As the spectrum of AI-based products and services is expanding at a rapid pace, there is an increased demand for computing systems that can efficiently handle various types of DNN workloads. Toward this, Ref. [265] presents an AI system design, RaPID, that integrates different approximation techniques developed at different layers of the computing stack. At the algorithm level, it employs precision scaling and gradient compression to reduce the compute, memory, and data-transfer costs. At the hardware level, it employs a reconfigurable approximate AI core that supports different levels of approximations. Finally, it incorporates an approximation-aware compiler to map approximate algorithms on approximate hardware. The system is designed to support various types of DNNs and both DNN inference and training workloads.

Of particular relevance, the need for further hardware efficiency across multiple abstraction levels also exists in resource-limited scenarios, suited to low-power ML, such as flexible and printed applications [18, 178]. Printed classifiers are significantly less costly than conventional CMOS technologies, paving the way for high circuit customizations, also known as Bespoke implementations [44]. Recent examples introduced Approximate Computing in printed ML classifiers tailored to Bespoke architectures [19, 20]. Ref. [20] proposes an automated framework that applies a model-to-circuit cross approximation and generates close-to-accuracy-optimal approximate ML circuits, allowing the deployment of complex classifiers on battery-powered devices (< 30mW). On

the other hand, through a hardware-friendly retraining (at algorithmic level), [19] replaces coefficients with more area-efficient ones, while an approximate neuron (at hardware-level) boosts the area/power efficiency by discarding the least significant information among the least significant summands.

4 Architectural Approximation Techniques

Architectures that offer the flexibility to willingly tradeoff among resource usage, storage and correctness in processor designs have the potential to achieve both higher performance and improved energy efficiency compared with error-free baselines. This section elaborates how these gains can be accomplished through the adoption of *approximate processors* and *approximate data storage*, where certain computational and/or storage tasks can be loaded to specialized/approximate computing units, resulting in faster processing at the expense of reduced accuracy [11, 57, 154, 173, 285]. Such approximations are typically implemented at a higher level of abstraction and involve reorganizing the system's functional blocks or optimizing the use of memory, rather than making modifications to the system's physical components (like in hardware approximation techniques).

4.1 Approximate Processors

Early research in the design of approximate processors includes a complete system and toolchain, proposed in Ref. [11]. The design spans from circuits to a compiler, which integrates an area- and energy-efficient analog neural accelerator tailored to approximate general-purpose code. This solution includes a compiler workflow that configures both the topology and weights of the neural network, while a customized training algorithm reduces errors derived from analog range limitations. In a similar approach [57], digital **Neural Processing Units** (**NPUs**) are tightly integrated with the processor pipeline, resulting in low-power approximations for specific sections of general-purpose code. Finally, approximate neural network processing along with high data-level parallelism has been explored for GPUs [285] and FPGAs [173], as well.

Relax [53] is an architectural framework that comprises three key components—ISA extension, hardware support and software support. The ISA extension allows the registration of a fault handler for a code region, enabling software applications to incorporate try/catch behavior. The hardware support for Relax simplifies hardware design and enhances energy efficiency by relaxing reliability constraints and providing fault detection mechanisms, while the software support offers a language-level recovery construct in C/C++, with relax blocks marking regions susceptible to hardware faults. Similarly, ERSA [48] is a system architecture that targets applications with inherent error resilience and ensures high degrees of resilience at low cost. It achieves high error resilience to high-order bit errors and control flow errors using asymmetric reliability (and the concept of "configurable reliability" for general-purpose applications) in many-core architectures, error-resilient algorithms, and intelligent software optimizations.

Prior research studies on the design of approximate processors also include the voltage overscaling paradigm as a potential energy-saving method with adjustable approximation level [3]. The authors in Ref. [56] propose an ISA extension provided with approximate operations and storage, as well as a microarchitecture design, called Truffle, which effectively supports these extensions. This work utilizes dual-voltage operation to save power and allows the programmers to specify which parts of a program can be computed approximately at a lower energy cost. Similar manipulations in operational (timing) errors to decrease power consumption of processor architectures from voltage/reliability tradeoffs, have been studied and can be found in Refs. [107, 229].

Among previous architectures, RISC-V has been established as an increasingly growing competitor and alternative of CPUs with funded projects and support from star companies (e.g., Intel,

Microsoft, ST Microelectronics [240]). The availability of open-source RISC-V tools and libraries provides developers with greater flexibility in designing and optimizing approximate computing systems. More specifically, the authors in Ref. [180] extend prior works in variable bit-width approximate arithmetic operators with configurable data memory units as well, while [115] leverages the inherent "value similarity" characteristic of ML workloads and by using lightweight microarchitectural ISA extensions skips entire instruction sequences. Accordingly, it substitutes the results with previously executed computations, improving both performance and energy efficiency. Furthermore, [61] utilizes multiple hardware-level approximations and minimizes quality degradation of several applications by adopting a hardware controller, which adjusts the degree of approximation by selecting various approximate units. Lastly, RISC-V processors targeting DNN applications have also been explored in Refs. [16, 49, 183]. In Ref. [183], the authors exploit the efficacy of SIMD instructions and propose mixed-precision **Quantized Neural Network** (**QNN**) execution that enables functional units down to 2 bits, opposed to commercial cores (e.g., ARM Cortex M55) that cannot natively support smaller than 8-bit SIMD instructions. On the other hand, both [49] and [16] extend the base RISC-V ISA to enable low bit-width and mixed-precision operations tailored to DNN tasks, but this time by modifying the core's processing unit that can be integrated in a full RISC-V core with minimal overheads.

4.2 Approximate Data Storage

This survey considers the two main types of memories: the non-volatile and the volatile memories. The works on approximate non-volatile data storage target **Phase-Change Memories** (**PCMs**) [60, 228, 252], flash memories [144, 222, 255], and **Spin Transfer Torque** (**SST**) RAMs [210, 225, 252, 291]. Regarding the volatile memories, the review includes static RAMs (SRAMs) [35, 63, 121, 213], dynamic RAMs (DRAMs) [105, 145, 204, 213], and caches [138, 159, 160]. It is noted that the literature also includes approximation techniques that are applied and evaluated in different types of memory [208].

4.2.1 Approximate Non-Volatile Memories. Sampson et al. [228] introduce two mechanisms to approximate PCMs, which can be used for persistent storage (e.g., for databases or filesystems) or as main memory. The first mechanism reduces the number of programming pulses for writing the multi-level cells (i.e., cells storing multiple bits of information), while the second one stores approximate data in blocks with exhausted error correction resources. SoftPCM [60] integrates a method to reduce the write traffic toward improved energy efficiency and lifetime. More specifically, a circuit compares the new data with stored data, and the writing operation is not performed if the difference is below a threshold.

The authors of Ref. [222] provide approximate storage in embedded flash memories by lowering the voltage below its nominal value. To deal with unpredictable behaviors and large error rates, they employ three software-based methods, i.e., "in-place writes", "multiple-place writes", and "RS-Berger codes". The first method repeatedly writes the data to the same memory address, while the second one writes the data in more than one addresses. The last method is an error detection and correction code that recovers the originally stored data. In the same context, Tseng et al. [255] characterize various multi-level cell flash memories when the supply voltage drops during read, program and erase operations. Based on the characterization results, they propose a dynamic scaling mechanism that adjusts the voltage supply of flash according to the executed operation. Retention relaxation [144] is another interesting technique applied in multi-level cell flash memories. In particular, the write operation for data with short lifetime (e.g., data from proxy and MapReduce workloads) is accelerated based on modeling the relationship between raw bit error rates and retention time.

The literature also includes works on approximate SST RAMs, which are non-volatile alternative of SRAMs for on-chip scratchpad or cache memory. Sampaio et al. [225] propose an approximate STT-RAM cache architecture that reduces the reliability overhead under a given error bound. The application's critical data are protected using a latency-aware double error correction module, while approximation-aware read and write policies provide approximate storage. Moreover, the architecture is equipped with a unit for monitoring the quality of the application's output. To provide an approximate scratchpad memory, Ranjan et al. [210] design a quality configurable SST-RAM, in which read and write operations are executed at varying accuracy levels. Approximate reads are performed by either lowering the current used to sense the stored value or lowering the sensing duration and increasing the current. Similarly, approximate writes are performed by lowering the current and/or sensing duration. Furthermore, the authors of Ref. [291] propose a scheme for progressively scaling the cells of STT-RAM arrays based on a scaling factor. In this approach, the most significant bits of the data words are implemented to provide lower bit error rates from the least-significant ones. In Ref. [252], the authors propose AdAM, which provides adaptive approximation management across the entire non-volatile memory hierarchy. In more detail, they model a system with STT-RAM scratchpad and PCM main memory and apply various approximation knobs (e.g., read/write pulse magnitude/duration) to exchange data accuracy for improved STT-RAM access delay and PCM lifetime.

4.2.2 Approximate Volatile Memories. Regarding SRAM memories, significant research efforts have focused on lowering the supply voltage. In this context, Kumar et al. [121] apply system-level design techniques to reduce the SRAM leakage power. Their framework is constrained by a data-reliability factor, while they use a statistical/probabilistic setup to model soft errors and process variations. In Ref. [35], the authors propose a hybrid memory array for video processors. Their policy is to store the high-order bits of luminance pixels, i.e., those that are more sensitive to human vision, in robust 8T SRAM bit-cells, while the low-order bits are stored in 6T bit-cells. This approach allows to lower the supply voltage without significantly affecting the high-order bits. Frustaci et al. [63] evaluate a wide range of approximation techniques (bit dropping, selective write assist, selective error correcting code, voltage scaling) for dynamic management of the energy–quality tradeoff in SRAMs. Their analysis is based on measurements on a 28-nm SRAM testchip, and it also includes combinations of the examined techniques and different array/word sizes.

The authors of Ref. [213] use approximate SRAM and DRAM memories to implement sketches for similarity estimation. In more detail, based on theoretical analysis and simulation, they examine the error rates in unprotected and parity-protected memories and tune their voltage supply. Moreover, the work in Ref. [145] applies lower refresh rates in the DRAM portions storing the non-critical data. Similarly, Jung et al. [105] define reliable and unreliable memory regions, however, in their strategy they completely disable the DRAM's auto-refresh feature. The authors in Ref. [204] propose a quality-configurable approximate DRAM system and evaluate it along with an FPGA. Their design leverages characterized memory errors to partition the DRAM into a number of quality bins based on the frequency, location, and nature of bit errors in DRAM pages, while also taking into account the variable retention time of the DRAM cells. The data placement in the bins is performed with respect to data criticality, with the refresh rate serving as the quality control knob.

Besides SRAMs and DRAMs, the literature also includes research works on approximate caches. In Ref. [160], the authors propose the Doppelganger cache based on the approximate similarity among the cache values. In particular, they associate the tags of multiple similar blocks with a single data array entry, and hence, they reduce the cache memory footprint. In the same context, the authors of Ref. [159] propose the Bunker cache based on the spatio-value similarity, where

Table 3. Quantitative Analysis of Software-Level Approximation Techniques for Deep Learning Workloads

Approximation Class	Workload	Metrics/Improvements	Accuracy Gain/Drop	Dependency on Other Layers	Ref.
Structured Pruning	ResNet50 on ImageNet dataset	63.5% reduction in FLOPs, 64.5% reduction in no. of parameters	1% drop in top-1 and 0.4% in top-5 accuracy	No	[43]
Unstructured Pruning	VGG16 on ImageNet dataset	79% reduction in FLOPs, 92.5% reduction in no. of parameters, 97.95% reduction in model size (with compression)	0.33% increase in top-1 and 0.41% in top-5 accuracy	Improves inference latency and energy efficiency when hardware-aware	[78]
Binarization	ResNet-18 on ImageNet dataset	32× model size reduction, 58× speed up in convolution (excluding memory allocation and access operations)	18.4% drop in top-1 and 16% in top-5 accuracy	Existing hardware supports required binary operations	[211]
Mixed Precision	MobileNet-V1/V2 on ImageNet dataset	1.4× to 1.95× latency reduction	Less than 1% accuracy loss	Yes, requires specialized hardware	[274]
Conditional Deep Learning Networks	ResNet-50 on Tiny ImageNet	6.8× reduction in no. of operations, 4.96× reduction in energy	1.12% accuracy improvement	No	[184]

Table 4. Quantitative Analysis of Hardware-Level Approximation Techniques for Deep Learning Workloads

Approximation Class	Workload	Metrics/Improvements	Accuracy Gain/Drop	Dependency on Other Layers	Ref.
Low-Precision DNNs with Error Compensation	ResNet50 on ImageNet dataset	≈10% energy reduction compared to 8-bit fixed-point	0.16% increase in accuracy compared to 8-bit FxP	No	[96]
Adaptive Numerical Data Type	ResNet on ImageNet dataset	On average 2.8× speedup and 2.5× energy efficiency improvement over other quantization accelerators	less than 1% drop in accuracy	No runtime dependencies	[72]
Voltage Scaling in DNN Accelerators	ResNet-18 on ImageNet dataset	47% power saving	Less than 2% accuracy loss	No	[98]
Inference using Approximate DRAM	VGG16 on ImageNet dataset	29% (on average) DRAM energy reduction	less than 1% accuracy drop	No runtime dependencies	[118]

approximately similar data exhibit spatial regularity in memory. Their approach is to map similar data to the same cache location with respect only to their memory address. Finally, ASCache [138] is an approximate SSD cache that allows bit errors within a tolerable threshold, while avoiding unnecessary cache misses and guaranteeing end-to-end data integrity.

5 Comparative Quantitative Analysis of Approximation Techniques

This section reports notable numerical results and outcomes for the application-specific and architectural approximation techniques.

5.1 Application-Specific Approximation Techniques

Tables 3 and 4 present a quantitative analysis of some of the most prominent software-level and hardware-level techniques covered in Section 3. This comparison considers only the methods evaluated on the ImageNet (or a relatively complex) dataset. From each sub-category at the software and hardware level (in Table 2), one of the most prominent or state-of-the-art techniques is selected. The goal is to highlight the efficiency and accuracy, providing a comparison of the strengths and tradeoffs across sub-categories and levels. For instance, the tables show that structured pruning outperforms in model size reduction, while conditional deep learning and adaptive numerical data types provide superior energy efficiency.

Table 5. Quantitative Analysis of Remarkable Approximation Techniques for Processors

Platform	Key Technique	Precision	Resource Gains	Quality Loss	Ref.
GPU	scalable units with quality control	32-bit	2.1× energy reduction	2.5% quality loss	[285]
FPGA	configurable network's topology and weights	16-bit	2.8×–28× energy savings	<10% quality loss	[173]
FPGA	RISC-V with posit processing unit	8-/16-bit	10× speedup	<1% accuracy loss	[49]
ASIC 7nm	mixed-precision and multi-pumped units	2-/4-/8-bit	415–1470 GOPs/W	<1% accuracy loss	[16]
ASIC 22nm	mixed-precision QNN with SIMD	2-/4-/8-bit	200–600 GOPs/W	<1% accuracy loss	[183]

It is important to note that each technique has unique advantages, and often, methods from different sub-categories and levels can be combined to achieve higher overall benefits with better accuracy -efficiency tradeoffs. In this context, Ref. [67] demonstrates that applying various approximations across different modules of a deep learning inference system (based on ResNet-101 for ImageNet classification) can achieve approximately 70% energy reduction with only a 0.77% absolute accuracy loss.

5.2 Architectural Approximation Techniques

Table 5 summarizes the key state-of-the-art works in the domain of approximate processors. In GPUs, approximate processing emphasizes scalability across multiple cores, integrating quality control mechanisms that reduce accuracy loss to just 2.5% while achieving over 2× energy efficiency [285]. For FPGAs, the reconfigurability enables substantial gains, delivering up to 28× speedup with medium-precision operations. While both FPGA and RISC-V cores offer significant advantages [173], ASIC-based designs demonstrate the most potential. Mixed-precision cores, capable of operating with as low as 2-bit precision, achieve performance levels between 600 [183] and 1470 [16] GOPs/W, with minimal accuracy loss. Especially when prioritizing efficiency over accuracy (e.g., allowing up to 5% degradation), RISC-V processors can achieve peak performance of up to 1.9 TOPs/W. In general, ASICs provide the optimal balance between energy efficiency and quality preservation, particularly in mixed-precision applications. Table 6 reports the corresponding analysis for approximate data storage with respect to the memory type. Overall, the state-of-the-art techniques mainly involve voltage scaling, approximate memory cells, and storage bins of different quality. Significant resource gains are achieved (e.g., up to 73% in DRAM power [204]), however, some techniques insert overheads (e.g., Ref. [255] in performance).

6 Application Domains of Approximate Computing

This section introduces the error-tolerant application domains that have been systematically examined for Approximate Computing. Table 7 classifies per domain the works presented in Part I [132] and Part II of the survey, while approximation opportunities and representative techniques for each domain are discussed. Furthermore, the case studies of each domain are analyzed, and widely used benchmark suites and error / **Quality-of-Service** (**QoS**) metrics are reported.

6.1 The Application Spectrum of Approximate Computing

Image Processing: This vital process is useful in various emerging applications from fields such as multimedia, bio-informatics, and robotics, which often require processing of a tremendous amount of data with diverse standards and formats [10]. Fortunately, image processing workloads exhibit

Table 6. Quantitative Analysis of Remarkable Memory Approximation Techniques

Memory	Main Techniques	Resource Gains	Loss / Overhead	Ref.
PCM	approx. multi-level cells, failed memory cells	1.7× speedup, +23% array lifetime	<10% quality loss	[228]
Flash	power fading, dynamic voltage scaling	45% energy	performance drop: avg=0.1%, worst=2%–19%	[255]
SST RAM	progressive cell scaling	37% power, 25% area	BER=10^{-6}	[291]
SRAM	bit dropping, voltage scaling, sel. write assist, sel. ECC	>2× energy	1% area	[63]
DRAM	quality bins construction	up to 73% power	~0% quality loss	[204]
Cache	spatio-value similarity	up to 1.2× speedup, up to 1.4× energy	small quality loss	[159]

Table 7. Classification of State-of-the-Art Approximate Computing Works Per Application Domain

Application Domain	References
Image Processing	[2, 7, 8, 10, 26, 33, 35, 38–42, 53, 55–58, 60–62, 64, 74, 83, 84, 89, 93, 100–103, 106, 122, 130, 134, 136–138, 140, 145–148, 150, 152, 153, 159–162, 164, 166, 167, 171, 173, 180, 189, 190, 193, 194, 201, 204–206, 209, 219–221, 223–226, 231, 233, 234, 237, 245, 248, 256–259, 261–264, 276, 277, 282, 285, 286, 292, 293, 295, 301]
Signal Processing	[10, 21, 22, 26, 37, 46, 56–58, 73, 74, 86, 99, 107, 110, 111, 130, 135–137, 142, 143, 149, 150, 155, 173, 181, 182, 189, 193, 200, 206, 207, 209, 216, 217, 223, 227, 228, 233, 244, 264, 270, 271, 282, 282, 287, 294, 295]
Computer Vision	[1, 22, 53, 89, 111, 140, 161, 164, 166, 167, 170, 182, 230, 237, 239, 248]
Computer Graphics	[21, 22, 30, 53, 56, 112, 145, 150, 163, 189, 190, 227, 228]
Big Data Analysis	[4, 12, 69, 90, 103, 108, 120, 127, 144, 192, 198, 199, 255, 279, 299]
Databases	[4, 21, 29, 108, 237, 255, 299]
Data Mining	[1, 11, 14, 22, 27, 29, 34, 47, 48, 53, 57, 89, 117, 134, 140, 148, 150, 156, 157, 159, 160, 170, 173, 201, 204, 205, 208–210, 212, 213, 224, 226, 229, 235, 237, 242, 248, 253, 282]
Machine Learning	[11, 19–21, 25, 47, 48, 57, 83, 109, 115–117, 159, 160, 166, 170, 173, 181, 182, 192, 201, 204, 205, 208–210, 223, 224, 229, 254, 282, 298]
Artificial Neural Networks	[3, 9, 13, 16, 30, 43, 48, 49, 52, 59, 65, 67, 67, 68, 71, 72, 78, 78–82, 85, 87, 88, 91, 92, 96, 97, 97, 98, 113, 114, 114, 118, 118, 128, 131, 133, 134, 139, 141, 151, 175–177, 183, 184, 186, 187, 194, 195, 201–204, 208, 210, 211, 218, 220, 229, 236, 238, 241, 246, 249–251, 258, 260, 265, 266, 268, 269, 272–275, 278, 282, 288–290, 297, 298, 300]
Scientific Computing	[14, 26, 27, 32, 45, 46, 51, 56, 62, 73, 100, 103, 110, 111, 124–126, 134, 158, 160, 162, 163, 189, 190, 206, 214, 216, 217, 227–230, 285, 287]
Financial Analysis	[1, 21, 22, 27, 62, 89, 103, 140, 142, 150, 160–162, 173, 223, 226, 235, 237, 256, 263, 285, 287, 296]
Physics Simulations	[1, 14, 28, 30, 53, 140, 150, 158, 160, 161, 163, 165, 167, 214, 215, 223, 226, 229, 263, 287, 296]

tolerance against errors, as most systems can accept minor perturbations or distortion within a quality degradation limit. Thus, Approximate Computing has received increasing attention here to mitigate the high computation cost and enhance the efficiency of the deployed systems by reasonable tradeoffs based on constraints of the specific application. A noticeable amount of Approximate Computing schemes and methodologies have focused on the filters inside the image processing function, which are applied to the input image to perform specific processing tasks, such as edge detection or morphological processing [58]. Most of them require intensive matrix computations and convolutions. Therefore, several works propose novel approximate functional units to replace the exact ones [8].

Signal Processing: In edge computing and IoT systems, various signal processing methodologies and algorithms are applied to process raw signal from sensors, which are often associated with noise [228]. Recently, signal processing algorithms are migrating from cloud-based platforms with high computation capabilities, such as servers and desktops, to the edge and low power-devices (i.e., embedded systems) that are often resource-constrained and cost-sensitive. Real-time requirements are common in signal processing systems, thus Approximate Computing schemes are proposed to suffice critical performance requirements [22].

Computer Vision: Computer vision is critical in many emerging applications such as augmented/virtual reality, autonomous driving and robotics. These applications often require intensive computation efforts on a large volume of data. One core component of a computer vision system is the computation kernel known as meta-functions, which are dedicated to execute one specific algorithm. As those meta-functions are typically inherently error resilient, voltage scaling techniques are applied to obtain attractive quality–energy tradeoffs [170]. Another application, ray tracing, which is also a common computer vision task, requires intensive arithmetic with floating-point numbers, whose precision can be formulated as a stochastic optimization problem for acceleration [230].

Computer Graphics: Modern digital photography and filming as well as 3D modeling, rendering and simulation depend on extensive support from computer graphics algorithms. These workloads demand a significant amount of computations due to the visual contents that are increasingly complex. Although computationally intensive, computer graphics show opportunities for Approximate Computing by having a strong data locality, which indicates that there are high redundancies that can be harvested to mitigate the computation costs [112]. By using cache to memorize outcomes of instructions, computer graphics applications such as OpenGL games can have a reduced number of instructions and witness acceleration [112].

Big Data Analysis: The data query is the first and foremost task in the big data analysis domain. Targeting improved performance, an approximate query engine BlinkDB is proposed in Ref. [4]. BlinkDB uses a set of stratified samples for the aim of optimization and deploys a dynamic sampling strategy to select an appropriately sized sample, which permits users to trade accuracy for response time. Other works are mainly based on Big Data frameworks such as Hadoop and Spark. The authors in Ref. [69] propose ApproxHadoop, which is a framework to run approximation-enabled MapReduce programs. ApproxHadoop leverages statistical theories to optimize the MapReduce programs when approximating with data sampling and task dropping.

Databases: Considering the Approximate Computing in database access, [108] introduces a novel data query system that injects samplers in the run-time and without requiring former existing samples. This system allows multiple join inputs and approximates the answer according to complex outputs. In the meantime, inspired by parallel computing, [237] utilizes the nature of approximation to improve the performance and decrease the error according to the accuracy demands.

Data Mining: During the data mining process, redundant computations, e.g., calling functions with the same parameters or using iterative input and output data, increase the execution time and lead to low performance. Ref. [27] targets this redundant computing and proposes an approximate approach named ATM. ATM employs both dynamic redundancy and approximation for a task-level computing that utilizes the previous task for the future. In practice, ATM also introduces an adaptive algorithm in the run-time that dynamically balances the tradeoff between accuracy and performance. Moreover, [22] uses the core-level redundancy, which is a combination of accurate and approximate versions of a task on different cores to increase the reliability.

Machine Learning: With the development of cloud computing, an increasing number of machine learning systems have become dominant in many sophisticated and crucial software

architectures. The machine learning models can be either supervised or unsupervised, learn features from the training data and deploy inference on unseen query data. Nowadays, most machine learning models are data-driven, meaning that the key to their success is the input data. However, in the cloud computing environment, the large scale data makes the machine learning process tardiness and energy-consuming. To reduce resource consumption, recent works propose approaches in different domains. The work in Ref. [21] presents an energy-efficient Approximate Computing framework that dynamically makes approximation decisions based on a predefined accuracy level. Moreover, Ref. [109] employs a run-time resource management approach based on functionally approximate kernels that exploit accuracy -performance tradeoffs within error resilient applications. This approach makes performance-aware decisions on power management under varied situations.

Artificial Neural Networks: Considering the number of arithmetic operations in **Artificial Neural Networks** (**ANNs**), the conventional architectures perform over millions of computations on each input data, leading to a high rate of timing and energy consumption. Therefore, as explained in Section 3, approximation techniques at different design layers aim at providing sufficient performance and power efficiency. The approximation of ANNs imposes several challenges. One of the them is to avoid retraining for reducing the accuracy loss [177], while the approximation localization [133] is also a key factor toward improved resources gains. Finally, cross-layer and end-to-end approaches [185] further increase the resource savings by exploiting the error resilience of each layer/sub-system.

Scientific Computing: Due to the rapid development of the computing industry in recent decades, scientific computing has become much more powerful and precise. Natural science research has been greatly enhanced by the availability of new materials and methodologies as well as the development of new hardware. Though the specific tasks in scientific computing may vary, the underlying mathematical functions are similar (e.g., linear equations, nonlinear equations, eigenvalue problems, FFT analysis). To meet their computing demands, the community aims to develop more efficient, robust, and fast frameworks for scientific computations. In this context, many studies have examined the tradeoff between precision and performance. One of the ideas is to get a balance between them by tuning the floating-point precision into mixed precision. The implementation in Ref. [126] uses binary instrumentation and modification to build mixed-precision configurations, whereas [28] uses dynamic program information and temporal locality to customize floating-point precision for scientific computing.

Financial Analysis: Financial analysis is another area in which approximate computations can be useful. The internet and the digitalization of the financial industry have led to increased importance of computation in financial analysis, especially when analyzing large amounts of data and incorporating machine learning. With the advent of these technologies, quantitative finance models continue to develop. The use of approximate computations increases the efficiency of data analysis, as well as enables real-time processes. For example, Ref. [256] demonstrates a compiler-directed output-based approximate memoization function that provides speedup in two different financial analysis applications to pricing a portfolio of stock options and swaptions.

Physics Simulations: Apart from being a part of scientific computing, simulation of physical phenomena involves some more specific areas, such as hydrodynamics simulation, thermodynamics simulation, and optical simulation. Many of these tasks are very compute-intensive. Approximate Computing can be applied to the physics simulation to accelerate this process. Ref. [167] presents a novel system, called OPPROX, for the application's execution-phase-aware approximation. OPPROX is demonstrated to be an efficient method for reducing dozens of workloads in a simulation of the Sedov blast wave problem.

6.2 Application-Driven Analysis of Case Studies

Table 8 presents the respective analysis of the aforementioned application domains, including also some statistical results about the dominant sub-categories for each layer of the computing stack. Regarding the distribution of the main techniques in the various research areas, Table 8 reveals a considerable heterogeneity that is contingent upon the particular use case. As shown, works in image processing are almost evenly distributed through the entire computing stack (i.e., 24 software, 26 hardware, and 26 architectural works). However, within the specific domain, notably, multiplier approximations dominate over the other hardware techniques, accounting for almost 80% of the total works. Similar observations arise for big data analysis, where software-oriented approximations this time, and specifically data sampling sub-category, constitute the vast majority and the most prominent solutions. On the other hand, it is noteworthy that in five application domains (computer graphics, big data analysis, databases, financial analysis, and physics simulations), there are no works focusing on hardware-level approximations. This might be explained by the fact that research and development efforts in these domains are traditionally revolved around software algorithms, optimizations and data management techniques, rather than the lower level of the design abstraction hierarchy. The latter highlights, thus, the space for further exploration and development of hardware-oriented approximate designs.

Table 8 also reports some significant experimental results, most of them concerning speedup, area and power/energy gains over conventional designs, with their respective overheads and/or quality losses. Out of all the techniques reported, it is observed that software-based approximations realize the highest speedups compared with hardware and architectural approximations. For example, the memoization approach of Ref. [27] achieves 16× speedup for only 5% memory overhead and 3% accuracy loss, while [299] manages up to 20× speedup for accurate execution. Finally, the table underscores the potential of Approximate Computing in machine learning applications, at each layer of the computing stack, that can attain remarkable energy reduction (22%–54%), while maintaining unaffected the level of quality with zero accuracy loss.

6.3 Benchmark Suites and Error Metrics

Various open source benchmark suites for Approximate Computing have been proposed and released. Table 9 reports some of the most well-established suites, which have been employed in the state-of-the-art works examined in this survey. Parallel programming benchmarks including **Princeton Application Repository for Shared-Memory Computers** (**PARSEC**) [24] and SPLASH-2 (Stanford Parallel Applications for Shared Memory) [280] are cross-domain benchmark suites with different emerging applications for evaluating and studying parallel chip-multiprocessors. There are also benchmarks focusing on multimedia and communication applications such as MediaBench [129]. SciMark is a benchmark suite primarily focusing on Java language applications of scientific computing, including kernels such as FFT, sparse matrix multiplication, Monte Carlo simulation and successive over-relaxation [197]. With the development of multi-platform parallel or acceleration libraries and frameworks such as OpenMP, OpenCL, and CUDA, more recent benchmarks focus on them. Rodinia is a benchmark set for heterogenous computing with diverse applications targeting at platforms from multi-core CPU systems to GPUs [36]. Regarding hardware, AxBench provides Verilog and C benchmark implementations [283].

The main error/QoS metrics that are used in Approximate Computing are presented in Table 10. At the bottom level, different error rates are used, such as **Error Rate** (**ER**), which calculates the number of erroneous results over the total number of results, and correspondingly, **Bit Error Ratio** (**BER**), which calculates the number of erroneous bits over the total bits. Another family of error metrics is the **Error Distance** (**ED**), including **Hamming Distance** (**HD**), Mean (Relative)

Table 8. Remarkable Improvements among All Examined Works with Respect to the Application Domain and the Different Layers of the Computing Stack

Application Domain	Stack	#Works	Main Techniques	Remarkable Gains	Ref.
Image Processing	SW	24	Approx. Progr. Language	34% energy gains and 113× less programming effort	[190]
	HW	26	Multiplier Approx.	95% energy gains and 85% area reduction for 1% accuracy loss	[258]
	Arch	26	Approx. Data Storage	1.7× speedup for <10% quality loss	[228]
Signal Processing	SW	13	Precision Scaling	1.33× speedup for no accuracy loss	[110]
	HW	11	Approx. Synthesis	60% power gains for <2% avg. error	[270]
	Arch	4	Approx. Processor	2.3× speedup and 3× energy gains for <10% accuracy loss	[57]
Computer Vision	SW	12	Loop Perforation	35% energy gains and 40% area reduction	[22]
	HW	3	Approx. Synthesis	23% power gains and 19% area savings	[182]
	Arch	1	Approx. Processors	40% energy gains for negligible quality loss	[53]
Computer Graphics	SW	10	Approx. Progr. Language	2×–17× energy gains	[189]
	HW	0	–	–	–
	Arch	3	Approx. Processor	up to 43% energy gains	[56]
Big Data Analysis	SW	13	Data Sampling	up to 8× speedup and 38% wait time reduction	[12]
	HW	0	–	–	–
	Arch	2	Approx. Data Storage	1.8×–5.7× speedup for <1% memory overhead	[144]
Databases	SW	5	Data Sampling	up to 20× speedup for precise execution	[299]
	HW	0	–	–	
	Arch	2	Approx. Data Storage	22%–45% energy gains for 0.1% performance loss	[255]
Data Mining	SW	19	Loop Perforation	6.4×/7.1× for 5%/10% error budget	[140]
	HW	6	Multiplier Approx.	up to 86% power, 46% delay and 63% area gains for NMED <0.0081	[148]
	Arch	15	Approx. Processor	1.2× speedup, 1.4× energy gains for 6% quality loss	[57]
Machine Learning	SW	12	Computation Skipping	2.18× energy gains for <0.5% accuracy loss	[205]
	HW	11	Voltage Over-Scaling	22% power reduction for no accuracy loss	[20]
	Arch	11	Approx. Processor	24% energy reduction for no accuracy loss	[180]
Artificial Neural Networks	SW	41	Computation Skipping	2× speedup, 1.9× energy gains for 3% accuracy loss	[9]
	HW	42	Multiplier Approx.	2.8× power gains for 5% accuracy loss	[134]
	Arch	11	Approx. Data Storage	up to 2× energy gains for negligible accuracy loss	[208]
Scientific Computing	SW	26	Approx. Progr. Language	up to 1.25× energy gains for negligible quality loss	[162]
	HW	2	Divider Approx.	3.9× speedup, 4.8× power gains for <5% NMED	[100]
	Arch	6	Approx. Data Storage	up to 2.55× energy gains, 1.55× cache area reduction for <10% quality error	[160]
Financial Analysis	SW	19	Approx. Memoization	16× speedup for 5% memory overhead and <3% accuracy loss	[27]
	HW	0	–	–	
	Arch	4	Approx. Processor	2.1× energy gains for 4% quality loss	[214]
Physics Simulations	SW	16	Approx. Memoization	3× speedup for 10% quality loss	[223]
	HW	0	–	–	
	Arch	5	Approx. Processor	1.25× energy gains for negligible quality loss	[53]

Table 9. Open-Source Benchmark Suites Used in State-of-the-Art Works of Approximate Computing

Benchmark Suite	Source Code	Application Domain
PARSEC [24]	C/C++	Image Processing, Computer Vision, Data Mining, Financial Analysis, Physics Simulations
SPLASH-2 [280]	C	Scientific Computing, Computer Graphics, Physics Simulations
MediaBench [129]	C	Image Processing, Signal Processing
MiBench [75]	C	Image Processing, Signal Processing
MiniBench [179]	C/C++	Data Mining
Rodinia [36]	OpenMP, OpenCL, CUDA	Data Mining, Scientific Computing, Physics Simulation, Machine Learning
PolyBench [196]	C, CUDA	Scientific Computing
SciMark 2.0 [197]	Java	Scientific Computing
AxBench [283]	C++, CUDA, Verilog	Image Processing, Signal Processing, Computer Graphics, Data Mining, Financial Analysis

Table 10. Error/QoS Metrics of Approximate Computing

Metric	Description	Application Domain
ER	Error Rate – Erroneous results per total results	General Computing
EP	Error Probability – Probability of error occurrence	
MED/MRED	Mean (Relative) Error Distance	
NMED/NMRED	Normalized Mean (Relative) Error Distance	
Max ED/RED	Maximum (Relative) Error Distance	
PED/PRED	Probability of (Relative) Error Distance > X	
MSE	Mean Squared Error	
RMSE	Root Mean Squared Error	
HD	Hamming Distance	
BER	Bit Error Ratio – Bit errors per total received bits	Digital Systems, Telecommunications
PER	Packet Error Ratio – Incorrect packets per total received packets	
PSNR	Peak Signal-to-Noise Ratio – Quality measurement between images	Image Processing, Video Processing, Computer Vision
SSIM	Structural Similarity – Quality measurement between images	
MPD	Mean Pixel Difference	
Classif. Accuracy	Correct classifications per total classifications	Pattern Recognition, Information Retrieval, Machine Learning
Precision	Relevant instances per total retrieved instances	
Recall	Relevant instances per total relevant instances	

ED (MED/MRED), Normalized MED (NMED), and other numerical-based metrics. Another metric that is employed in the evaluation of approximate designs is the Probability of (Relative) ED higher than X (PRED/PRED), where X is a numerical value, Finally, **Mean Squared Error** (**MSE**) and **Root MSE** (**RMSE**) are also common metrics to evaluate the output quality.

There are also application-specific error metrics to describe the efficiency of the approximations at a higher level. For example, in telecommunications, **Packet Error Ratio** (**PER**) is usually considered to evaluate the communication quality. In ML applications, pattern recognition and information retrieval, classification accuracy is usually considered, as well as other metrics such as precision (relevant instances of retrieved instances) and recall (relevant instances of total instances).

7 Challenges and Future Directions

This section highlights several key challenges that need to be addressed to accelerate the progress in Approximate Computing and enable its use in real-world systems.

Cross-Layer and End-to-End System-Level Approximations: Most of the research in Approximate Computing has focused on individual abstraction layers of the computing stack. However, the true potential of Approximate Computing lies in cross-layer and end-to-end

system-level approximations, where multiple complementary approximation techniques are combined to achieve a better quality–efficiency tradeoff. Therefore, a system-level view is necessary for trading quality for efficiency, as approximation can be deployed at different abstraction layers and in different components. Only the right set of approximations deployed at the right layers and in the right components can offer optimal returns. Some existing works in this direction include [5, 67, 85]. However, still a significant amount of research is required to develop sophisticated automated methodologies that can efficiently identify the appropriate set of approximations for all the components in a system to offer the best quality–efficiency tradeoff while satisfying the user-defined constraints. One of the key limitations toward enabling such methodologies is the lack of tools and methods that can efficiently and accurately quantify the impact of approximations, specifically multiple types of approximations, on the application-level accuracy of the systems.

Hardware Models: The field of Approximate Computing lacks validated hardware models that are essential for performing efficient-yet-accurate design space exploration to select appropriate level of approximation for each hardware module in the system. For example, the models used for voltage overscaling are based on assumptions about the distribution of errors in computing and memory structures. Similarly, most of the ReRAM-based in-memory computing works are based on assumptions about the precision with which values can be written to or read from the memory arrays. Moreover, different works use different models and assumptions, which makes it difficult to compare techniques based on the reported results. Therefore, there is a dire need for building hardware models that have been validated by the hardware community through extensive simulations. The models need to be validated for a wide variety of scenarios to enable their reliable deployment in cross-layer approximation methodologies.

Benchmarks: The Approximate Computing community lacks representative sets of applications as well as datasets from different application domains that can be used for benchmarking approximation techniques. Having a carefully selected set of applications and datasets is crucial, as the impact of approximations can vary drastically from application to application and even from data sample to data sample. Some efforts have been made toward defining benchmarks, e.g., AxBench [284]; however, these efforts cover only a small set of applications and scenarios. Therefore, there is a need for a much broader set of applications and datasets from a wider spectrum of application domains. Moreover, as it is difficult to cover all the application domains and applications, specifically when new applications are emerging on a daily basis due to the rapid pace of technology growth, there is a need for standards and rules for defining evaluation strategies and selecting datasets for an unbiased and thorough evaluation of approximation techniques.

Evaluation Metrics: Design space exploration methodologies are guided through quality and performance evaluation metrics to find the optimal configurations that offer the best quality–efficiency tradeoffs and meet all the user-defined constraints. As a result, it is crucial to have reliable quality and performance evaluation metrics in place. Metrics that do not capture suitable characteristics can lead to sub-optimal configurations, eventually affecting the user experience. Therefore, it is essential to define the right set of quality evaluation metrics for each application.

Post-Fabrication Testing: Process variations, which lead to permanent faults and variations in the hardware characteristics of transistors, are a major concern in nano-scale devices. All the fabricated devices are passed through post-fabrication testing to detect manufacturing-induced defects and variations. However, approximations make this already challenging process further challenging. Only a limited amount of work has been carried out toward testing of Approximate Computing devices. Therefore, there is a dire need for specialized approximation-aware testing methodologies that can enable efficient-yet-accurate testing of these devices and reduce the yield loss, wherever possible, through binning.

Reproducibility for Fair Comparison and Future Developments: In most cases, it is challenging to replicate the exact evaluation methodology and setup reported in research works, making it difficult (if not impossible) to reproduce the results. Therefore, it is essential to promote open-source contributions to ensure the reproducibility of the results. The open-source contributions will also facilitate further research and development in the area of Approximate Computing, resulting in rapid progress toward uncovering the true potential of Approximate Computing for improving the efficiency of computing systems.

8 Conclusion

This article presented Part II of a comprehensive survey on Approximate Computing, reviewing the state-of-the-art application-specific and architectural approximation techniques. Regarding the application-specific techniques, the survey focused on the emerging AI/ML applications and discussed about software, hardware, cross-layer and end-to-end approximations. Regarding the architectural techniques, the survey focused on approximate processors and accelerators and different memory types for approximate data storage. Moreover, it included an extensive analysis of the application spectrum of Approximate Computing, i.e., a domain classification of all the works reviewed, it presented representative techniques and remarkable results, as well as it reported well-established error metrics and benchmark suites for evaluating the QoS of approximate designs. Despite the notable progress demonstrated by Approximate Computing, there remains a pressing need for ongoing and significant innovation to fully realize the potential of approximations in the design of complex computing systems. These open challenges were discussed along with future directions at the end of the survey.

References

[1] Sara Achour and Martin C. Rinard. 2015. Approximate computation with outlier detection in Topaz. In *ACM SIGPLAN Int'l. Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'15)*. 711–730.

[2] Elizabeth Adams, Suganthi Venkatachalam, and Seok-Bum Ko. 2020. Approximate restoring dividers using inexact cells and estimation from partial remainders. *IEEE Trans. on Computers* 69, 4 (2020), 468–474.

[3] Hassan Afzali-Kusha and Massoud Pedram. 2023. X-NVDLA: Runtime accuracy configurable NVDLA based on applying voltage overscaling to computing and memory units. *IEEE Trans. on Circuits and Systems I: Regular Papers* 70, 5 (2023), 1989–2002.

[4] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. 2013. BlinkDB: Queries with bounded errors and bounded response times on very large data. In *ACM SIGOPS European Conference on Computer Systems (EuroSys)*. 29–42.

[5] Ankur Agrawal, Jungwook Choi, Kailash Gopalakrishnan, Suyog Gupta, Ravi Nair, Jinwook Oh, Daniel A. Prener, Sunil Shukla, Vijayalakshmi Srinivasan, and Zehra Sura. 2016. Approximate computing: Challenges and opportunities. In *IEEE Int'l. Conference on Rebooting Computing (ICRC)*. 1–8.

[6] Ankur Agrawal, Sae Kyu Lee, Joel Silberman, Matthew Ziegler, Mingu Kang, Swagath Venkataramani, Nianzheng Cao, Bruce Fleischer, Michael Guillorn, Matthew Cohen, Silvia Mueller, Jinwook Oh, Martin Lutz, Jinwook Jung, Siyu Koswatta, Ching Zhou, Vidhi Zalani, James Bonanno, Robert Casatuta, Chia-Yu Chen, Jungwook Choi, Howard Haynie, Alyssa Herbert, Radhika Jain, Monodeep Kar, Kyu-Hyoun Kim, Yulong Li, Zhibin Ren, Scot Rider, Marcel Schaal, Kerstin Schelm, Michael Scheuermann, Xiao Sun, Hung Tran, Naigang Wang, Wei Wang, Xin Zhang, Vinay Shah, Brian Curran, Vijayalakshmi Srinivasan, Pong-Fei Lu, Sunil Shukla, Leland Chang, and Kailash Gopalakrishnan. 2021. A 7nm 4-Core AI Chip with 25.6TFLOPS hybrid FP8 training, 102.4TOPS INT4 inference and workload-aware throttling. In *IEEE Int'l. Solid-State Circuits Conference (ISSCC)*, Vol. 64. 144–146.

[7] Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2017. Dual-quality 4:2 compressors for utilizing in dynamic accuracy configurable multipliers. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 25, 4 (2017), 1352–1361.

[8] Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2018. RAP-CLA: A reconfigurable approximate carry look-ahead adder. *IEEE Trans. on Circuits and Systems II: Express Briefs* 65, 8 (2018), 1089–1093.

[9] Vahideh Akhlaghi, Amir Yazdanbakhsh, Kambiz Samadi, Rajesh K. Gupta, and Hadi Esmaeilzadeh. 2018. SnaPEA: Predictive early activation for reducing computation in deep convolutional neural networks. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA)*. 662–673.

[10] Carlos Alvarez, Jesus Corbal, and Mateo Valero. 2005. Fuzzy memoization for floating-point multimedia applications. *IEEE Trans. on Computers* 54, 7 (2005), 922–927.
[11] Renée St. Amant, et al. 2014. General-purpose code acceleration with limited-precision analog computation. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA'14)*. 505–516.
[12] Michael R. Anderson and Michael Cafarella. 2016. Input selection for fast feature engineering. In *IEEE Int'l. Conference on Data Engineering (ICDE'16)*. 577–588.
[13] Mohammad Saeed Ansari, Bruce F. Cockburn, and Jie Han. 2021. An improved logarithmic multiplier for energy-efficient neural computing. *IEEE Trans. on Computers* 70, 4 (2021), 614–625.
[14] Jason Ansel, Yee Lok Wong, Cy Chan, Marek Olszewski, Alan Edelman, and Saman Amarasinghe. 2011. Language and compiler support for auto-tuning variable-accuracy algorithms. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization (CGO)*. 85–96.
[15] Alexander Aponte-Moreno, Alejandro Moncada, Felipe Restrepo-Calle, and Cesar Pedraza. 2018. A review of approximate computing techniques towards fault mitigation in HW/SW systems. In *IEEE Latin-American Test Symposium (LATS)*. 1–6.
[16] Giorgos Armeniakos, Alexis Maras, Sotirios Xydis, and Dimitrios Soudris. 2024. Mixed-precision neural networks on RISC-V cores: ISA extensions for multi-pumped soft SIMD operations. In *Int'l. Conference on Computer-Aided Design (ICCAD)*. 1–9.
[17] Giorgos Armeniakos, Georgios Zervakis, Dimitrios Soudris, and Jörg Henkel. 2022. Hardware approximate techniques for deep neural network accelerators: A survey. *Comput. Surveys* 55, 4 (2022), 1–36.
[18] Giorgos Armeniakos, Georgios Zervakis, Dimitrios Soudris, Mehdi B. Tahoori, and Jörg Henkel. 2022. Cross-layer approximation for printed machine learning circuits. In *Design, Automation and Test in Europe (DATE)*. 1–6.
[19] Giorgos Armeniakos, Georgios Zervakis, Dimitrios Soudris, Mehdi B. Tahoori, and Jörg Henkel. 2023. Co-design of approximate multilayer perceptron for ultra-resource constrained printed circuits. *IEEE Trans. on Computers* 72, 9 (2023), 2717–2725.
[20] Giorgos Armeniakos, Georgios Zervakis, Dimitrios Soudris, Mehdi B. Tahoori, and Jörg Henkel. 2023. Model-to-circuit cross-approximation for printed machine learning classifiers. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 42, 11 (2023), 3532–3544.
[21] Woongki Baek and Trishul M. Chilimbi. 2010. Green: A framework for supporting energy-conscious programming using controlled approximation. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 198–209.
[22] Farshad Baharvand and Seyed Ghassem Miremadi. 2020. LEXACT: Low energy N-Modular redundancy using approximate computing for real-time multicore processors. *IEEE Trans. on Emerging Topics in Computing* 8, 2 (2020), 431–441.
[23] Paolo Bertoldi, Maria Avgerinou, and Luca Castellazzi. 2017. Trends in data centre energy consumption under the european code of conduct for data centre energy efficiency. *European Comission—Joint Research Centre (JRC)* JRC108354 (2017), 1–43. https://publications.jrc.ec.europa.eu/repository/handle/JRC108354
[24] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In *Int'l. Conf. on Parallel Architectures and Compilation Techniques*. 72–81.
[25] James Bornholt, Todd Mytkowicz, and Kathryn S. McKinley. 2014. Uncertain<T>: A first-order type for uncertain data. In *ACM Int'l. Conference on Architectural Support for Programming Languages and Operating Systems*. 51–66.
[26] Brett Boston, Adrian Sampson, Dan Grossman, and Luis Ceze. 2015. Probability type inference for flexible approximate programming. In *ACM SIGPLAN Int'l. Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 470–487.
[27] Iulian Brumar, Marc Casas, Miquel Moreto, Mateo Valero, and Gurindar S. Sohi. 2017. ATM: Approximate task memoization in the runtime system. In *IEEE Int'l. Parallel and Distributed Processing Symposium (IPDPS)*. 1140–1150.
[28] Hugo Brunie, Costin Iancu, Khaled Z. Ibrahim, Philip Brisk, and Brandon Cook. 2020. Tuning floating-point precision using dynamic program information and temporal locality. In *ACM/IEEE SC, Int'l. Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
[29] Surendra Byna, Jiayuan Meng, Anand Raghunathan, Srimat Chakradhar, and Srihari Cadambi. 2010. Best-effort semantic document search on GPUs. In *Workshop on General-Purpose Computation on Graphics Processing Units (GPGPU)*. 86–93.
[30] Simone Campanoni, Glenn Holloway, Gu-Yeon Wei, and David Brooks. 2015. HELIX-UP: Relaxing program semantics to unleash parallelization. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization (CGO)*. 235–245.
[31] Zhiwei Cao, Xin Zhou, Han Hu, Zhi Wang, and Yonggang Wen. 2022. Toward a systematic survey for carbon neutral data centers. *IEEE Communications Surveys and Tutorials* 24, 2 (2022), 895–936.
[32] Michael Carbin, Sasa Misailovic, and Martin C. Rinard. 2013. Verifying quantitative reliability for programs that execute on unreliable hardware. In *ACM SIGPLAN Int'l. Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 33–52.

[33] Jorge Castro-Godínez, Julián Mateus-Vargas, Muhammad Shafique, and Jörg Henkel. 2020. AxHLS: Design space exploration and high-level synthesis of approximate accelerators using approximate functional units and analytical models. In *Int'l. Conference On Computer Aided Design (ICCAD)*. 1–9.

[34] Srimat T. Chakradhar and Anand Raghunathan. 2010. Best-effort computing: Re-thinking parallel software and hardware. In *Design Automation Conference (DAC)*. 865–870.

[35] Ik Joon Chang, Debabrata Mohapatra, and Kaushik Roy. 2011. A priority-based 6T/8T hybrid SRAM architecture for aggressive voltage scaling in video applications. *IEEE Trans. on Circuits and Systems for Video Technology* 21, 2 (2011), 101–112.

[36] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *IEEE Int'l. Symposium on Workload Characterization (IISWC)*. 44–54.

[37] Jienan Chen and Jianhao Hu. 2013. Energy-efficient digital signal processing via voltage-overscaling-based residue number system. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 21, 7 (2013), 1322–1332.

[38] Ke Chen, Linbin Chen, Pedro Reviriego, and Fabrizio Lombardi. 2019. Efficient implementations of reduced precision redundancy (RPR) multiply and accumulate (MAC). *IEEE Trans. on Computers* 68, 5 (2019), 784–790.

[39] Linbin Chen, Jie Han, Weiqiang Liu, and Fabrizio Lombardi. 2015. Design of approximate unsigned integer non-restoring divider for inexact computing. In *Great Lakes Symposium on VLSI (GLSVLSI)*. 51–56.

[40] Linbin Chen, Jie Han, Weiqiang Liu, and Fabrizio Lombardi. 2016. On the design of approximate restoring dividers for error-tolerant applications. *IEEE Trans. on Computers* 65, 8 (2016), 2522–2533.

[41] Linbin Chen, Jie Han, Weiqiang Liu, and Fabrizio Lombardi. 2017. Algorithm and design of a fully parallel approximate coordinate rotation digital computer (CORDIC). *IEEE Trans. on Multi-Scale Computing Systems* 3, 3 (2017), 139–151.

[42] Linbin Chen, Jie Han, Weiqiang Liu, Paolo Montuschi, and Fabrizio Lombardi. 2018. Design, evaluation and application of approximate high-radix dividers. *IEEE Trans. on Multi-Scale Computing Systems* 4, 3 (2018), 299–312.

[43] Tianyi Chen, Bo Ji, Tianyu Ding, Biyi Fang, Guanyi Wang, Zhihui Zhu, Luming Liang, Yixin Shi, Sheng Yi, and Xiao Tu. 2021. Only train once: A one-shot neural network training and pruning framework. *Advances in Neural Information Processing Systems* 34 (2021), 19637–19651.

[44] Hari Cherupalli, Henry Duwe, Weidong Ye, Rakesh Kumar, and John Sartori. 2017. Bespoke processors for applications with ultra-low area and power constraints. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA)*. 41–54.

[45] Wei-Fan Chiang, Mark Baranowski, Ian Briggs, Alexey Solovyev, Ganesh Gopalakrishnan, and Zvonimir Rakamarić. 2017. Rigorous floating-point mixed-precision tuning. In *ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 300–315.

[46] Wei-Fan Chiang, Ganesh Gopalakrishnan, Zvonimir Rakamaric, and Alexey Solovyev. 2014. Efficient search for inputs causing high floating-point errors. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 43–52.

[47] Vinay Kumar Chippa, Debabrata Mohapatra, Kaushik Roy, Srimat T. Chakradhar, and Anand Raghunathan. 2014. Scalable effort hardware design. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 22, 9 (2014), 2004–2016.

[48] Hyungmin Cho, Larkhoon Leem, and Subhasish Mitra. 2012. ERSA: Error resilient system architecture for probabilistic applications. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 31, 4 (2012), 546–558.

[49] Marco Cococcioni, Federico Rossi, Emanuele Ruffaldi, and Sergio Saponara. 2022. A lightweight posit processing unit for RISC-V processors in deep neural network applications. *IEEE Trans. on Emerging Topics in Computing* 10, 4 (2022), 1898–1908.

[50] Hans Jakob Damsgaard, Aleksandr Ometov, and Jari Nurmi. 2023. Approximation opportunities in edge computing hardware: A systematic literature review. *Comput. Surveys* 55, 12 (2023), 1–49.

[51] Eva Darulova and Viktor Kuncak. 2017. Towards a compiler for reals. *ACM Trans. on Programming Languages and Systems* 39, 2 (2017), 1–28.

[52] Arghadip Das, Soumendu Kumar Ghosh, Arnab Raha, and Vijay Raghunathan. 2024. Toward energy-efficient collaborative inference using multisystem approximations. *IEEE Internet of Things Journal* 11, 10 (2024), 17989–18004.

[53] Marc de Kruijf, Shuou Nomura, and Karthikeyan Sankaralingam. 2010. Relax: An architectural framework for software recovery of hardware faults. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA)*. 497–508.

[54] Cecilia De la Parra, Andre Guntoro, and Akash Kumar. 2020. ProxSim: GPU-based simulation framework for cross-layer approximate DNN optimization. In *Design, Automation and Test in Europe (DATE)*. 1193–1198.

[55] Farhad Ebrahimi-Azandaryani, Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2020. Block-based carry speculative approximate adder for energy-efficient applications. *IEEE Trans. on Circuits and Systems II: Express Briefs* 67, 1 (2020), 137–141.

[56] Hadi Esmaeilzadeh, Adrian Sampson, Luis Ceze, and Doug Burger. 2012. Architecture support for disciplined approximate programming. In *ACM Int'l. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 301–312.

[57] Hadi Esmaeilzadeh, Adrian Sampson, Luis Ceze, and Doug Burger. 2012. Neural acceleration for general-purpose approximate programs. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 449–460.

[58] Darjn Esposito, Antonio Giuseppe Maria Strollo, Ettore Napoli, Davide De Caro, and Nicola Petra. 2018. Approximate multipliers based on new approximate compressors. *IEEE Trans. on Circuits and Systems I: Regular Papers* 65, 12 (2018), 4169–4182.

[59] Yinghui Fan, Xiaoxi Wu, Jiying Dong, and Zhi Qi. 2019. AxDNN: Towards the cross-layer design of approximate DNNs. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*. 317–322.

[60] Yuntan Fang, Huawei Li, and Xiaowei Li. 2012. SoftPCM: Enhancing energy efficiency and lifetime of phase change memory in video applications via approximate write. In *IEEE Asian Test Symposium (ATS)*. 131–136.

[61] Isaías Felzmann, João Fabrício Filho, and Lucas Wanner. 2020. Risk-5: Controlled approximations for RISC-V. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 4052–4063.

[62] Vimuth Fernando, Keyur Joshi, and Sasa Misailovic. 2019. Verifying safety and accuracy of approximate parallel programs via canonical sequentialization. *Proceedings of the ACM on Programming Languages* 3, 119 (2019), 1–29.

[63] Fabio Frustaci, David Blaauw, Dennis Sylvester, and Massimo Alioto. 2016. Approximate SRAMs with dynamic energy-quality management. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 24, 6 (2016), 2128–2141.

[64] Fabio Frustaci, Stefania Perri, Pasquale Corsonello, and Massimo Alioto. 2020. Approximate multipliers with dynamic truncation for energy reduction via graceful quality degradation. *IEEE Trans. on Circuits and Systems II: Express Briefs* 67, 12 (2020), 3427–3431.

[65] Sanjay Ganapathy, Swagath Venkataramani, Giridhur Sriraman, Balaraman Ravindran, and Anand Raghunathan. 2020. DyVEDeep: Dynamic variable effort deep neural networks. *ACM Trans. on Embedded Computing Systems* 19, 3 (2020), 1–24.

[66] Georgios Georgis, George Lentaris, and Dionysios Reisis. 2019. Acceleration techniques and evaluation on multi-core CPU, GPU and FPGA for image processing and super-resolution. *Springer Journal of Real-Time Image Processing* 16, 4 (2019), 1207–1234.

[67] Soumendu Kumar Ghosh, Arnab Raha, and Vijay Raghunathan. 2020. Approximate inference systems (AxIS) end-to-end approximations for energy-efficient inference at the edge. In *ACM/IEEE Int'l. Symposium on Low Power Electronics and Design (ISLPED)*. 7–12.

[68] Soumendu Kumar Ghosh, Arnab Raha, and Vijay Raghunathan. 2023. Energy-efficient approximate edge inference systems. *ACM Trans. on Embedded Computing Systems* 22, 4 (2023), 1–50.

[69] Inigo Goiri, Ricardo Bianchini, Santosh Nagarakatte, and Thu D. Nguyen. 2015. ApproxHadoop: Bringing approximations to MapReduce frameworks. In *ACM Int'l. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 383–397.

[70] Jing Gong, Hassaan Saadat, Hasindu Gamaarachchi, Haris Javaid, Xiaobo Sharon Hu, and Sri Parameswaran. 2023. ApproxTrain: Fast simulation of approximate multipliers for DNN training and inference. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 42, 11 (2023), 3505–3518.

[71] Yu Gong, Bo Liu, Wei Ge, and Longxing Shi. 2019. ARA: Cross-layer approximate computing framework based reconfigurable architecture for CNNs. *Elsevier Microelectronics Journal* 87 (2019), 33–44.

[72] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. ANT: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 1414–1433.

[73] Hui Guo and Cindy Rubio-González. 2018. Exploiting community structure for floating-point precision tuning. In *ACM SIGSOFT Int'l. Symposium on Software Testing and Analysis (ISSTA)*. 333–343.

[74] Vaibhav Gupta, Debabrata Mohapatra, Anand Raghunathan, and Kaushik Roy. 2013. Low-power digital signal processing using approximate adders. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 32, 1 (2013), 124–137.

[75] Matthew R. Guthaus, Jeffrey S. Ringenberg, Dan Ernst, Todd M. Austin, Trevor Mudge, and Richard B. Brown. 2001. MiBench: A free, commercially representative embedded benchmark suite. In *IEEE Int'l. Workshop on Workload Characterization (WWC)*. 3–14.

[76] Jie Han and Michael Orshansky. 2013. Approximate computing: An emerging paradigm for energy-efficient design. In *IEEE European Test Symposium (ETS)*. 1–6.

[77] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A Horowitz, and William J Dally. 2016. EIE: Efficient inference engine on compressed deep neural network. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 243–254.

[78] Song Han, Huizi Mao, and William J. Dally. 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations (ICLR'16)*. 1–14.
[79] Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both Weights and Connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NIPS)*. 1135–1143.
[80] Muhammad Abdullah Hanif, Faiq Khalid, and Muhammad Shafique. 2019. CANN: Curable approximations for high-performance deep neural network accelerators. In *Design Automation Conference (DAC)*. 1–6.
[81] Muhammad Abdullah Hanif, Giuseppe Maria Sarda, Alberto Marchisio, Guido Masera, Maurizio Martina, and Muhammad Shafique. 2022. CoNLoCNN: Exploiting correlation and non-uniform quantization for energy-efficient low-precision deep convolutional neural networks. In *Int'l. Joint Conference on Neural Networks (IJCNN)*. 1–8.
[82] Muhammad Abdullah Hanif and Muhammad Shafique. 2022. A cross-layer approach towards developing efficient embedded deep learning systems. *Elsevier Microprocessors and Microsystems* 88 (2022), 103609.
[83] Soheil Hashemi, R. Iris Bahar, and Sherief Reda. 2015. DRUM: A dynamic range unbiased multiplier for approximate applications. In *Int'l. Conference on Computer-Aided Design (ICCAD)*. 418–425.
[84] Soheil Hashemi, R. Iris Bahar, and Sherief Reda. 2016. A low-power dynamic divider for approximate applications. In *Design Automation Conference (DAC)*. 1–6.
[85] Soheil Hashemi, Hokchhay Tann, Francesco Buttafuoco, and Sherief Reda. 2018. Approximate computing for biometric security systems: A case study on iris scanning. In *Design, Automation and Test in Europe (DATE)*. 319–324.
[86] Soheil Hashemi, Hokchhay Tann, and Sherief Reda. 2018. BLASYS: Approximate logic synthesis using boolean matrix factorization. In *Design Automation Conference (DAC)*. 1–6.
[87] Yihui He, Xiangyu Zhang, and Jian Sun. 2017. Channel pruning for accelerating very deep neural networks. In *IEEE Int'l. Conference on Computer Vision (ICCV)*. 1389–1397.
[88] Zhiqiang He, Yaguan Qian, Yuqi Wang, Bin Wang, Xiaohui Guan, Zhaoquan Gu, Xiang Ling, Shaoning Zeng, Haijiang Wang, and Wujie Zhou. 2022. Filter pruning via feature discrimination in deep neural networks. In *European Conference on Computer Vision (ECCV)*. 245–261.
[89] Henry Hoffmann, Sasa Misailovic, Stelios Sidiroglou, Anant Agarwal, and Martin C. Rinard. 2009. Using code perforation to improve performance, reduce energy consumption, and respond to failures. *Massachusetts Institute of Technology Technical Report* MIT-CSAIL-TR-2009-042 (2009), 1–21. https://dspace.mit.edu/handle/1721.1/46709
[90] Guangyan Hu, Sandro Rigo, Desheng Zhang, and Thu Nguyen. 2019. Approximation with error bounds in Spark. In *IEEE Int'l. Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*. 61–73.
[91] Hengyuan Hu, Rui Peng, Yu-Wing Tai, and Chi-Keung Tang. 2016. Network trimming: A data-driven neuron pruning approach towards efficient deep architectures. 1–9.
[92] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. *Advances in Neural Information Processing Systems* 29 (2016), 1–9.
[93] Mohsen Imani, Ricardo Garcia, Andrew Huang, and Tajana Rosing. 2019. CADE: Configurable approximate divider for energy efficiency. In *Design, Automation and Test in Europe (DATE)*. 586–589.
[94] IoT Analytics. 2023. *State of IoT 2023*. Retrieved from https://iot-analytics.com/number-connected-iot-devices/
[95] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
[96] Shubham Jain, Swagath Venkataramani, Vijayalakshmi Srinivasan, Jungwook Choi, Pierce Chuang, and Leland Chang. 2018. Compensated-DNN: Energy efficient low-precision deep neural networks by compensating quantization errors. In *Design Automation Conference (DAC)*. 1–6.
[97] Shubham Jain, Swagath Venkataramani, Vijayalakshmi Srinivasan, Jungwook Choi, Kailash Gopalakrishnan, and Leland Chang. 2019. BiScaled-DNN: Quantizing long-tailed datastructures with two scale factors for deep neural networks. In *Design Automation Conference (DAC)*. 1–6.
[98] Daehan Ji, Dongyeob Shin, and Jongsun Park. 2020. An error compensation technique for low-voltage DNN accelerators. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 29, 2 (2020), 397–408.
[99] Honglan Jiang, Jie Han, Fei Qiao, and Fabrizio Lombardi. 2016. Approximate radix-8 Booth multipliers for low-power and high-performance operation. *IEEE Trans. on Computers* 65, 8 (2016), 2638–2644.
[100] Honglan Jiang, Leibo Liu, Fabrizio Lombardi, and Jie Han. 2019. Low-power unsigned divider and square root circuit designs using adaptive approximation. *IEEE Trans. on Computers* 68, 11 (2019), 1635–1646.
[101] Xun Jiao, Dongning Ma, Wanli Chang, and Yu Jiang. 2020. LEVAX: An input-aware learning-based error model of voltage-scaled functional units. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 5032–5041.
[102] Xun Jiao, Dongning Ma, Wanli Chang, and Yu Jiang. 2020. TEVoT: Timing error modeling of functional units under dynamic voltage and temperature variations. In *Design Automation Conference (DAC)*. 1–6.

[103] Keyur Joshi, Vimuth Fernando, and Sasa Misailovic. 2019. Statistical algorithmic profiling for randomized approximate programs. In *ACM/IEEE Int'l. Conference on Software Engineering (ICSE)*. 608–618.

[104] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA)*. 1–12.

[105] Matthias Jung, Éder Zulian, Deepak M. Mathew, Matthias Herrmann, Christian Brugger, Christian Weis, and Norbert Wehn. 2015. Omitting refresh: A case study for commodity and wide I/O DRAMs. In *Int'l. Symposium on Memory Systems (MEMSYS)*. 85–91.

[106] Andrew B. Kahng and Seokhyeong Kang. 2012. Accuracy-configurable adder for approximate arithmetic designs. In *Design Automation Conference (DAC)*. 820–825.

[107] Andrew B. Kahng, Seokhyeong Kang, Rakesh Kumar, and John Sartori. 2010. Designing a processor from the ground up to allow voltage/reliability tradeoffs. In *IEEE Int'l. Symposium on High Performance Computer Architecture*. 1–11.

[108] Srikanth Kandula, Anil Shanbhag, Aleksandar Vitorovic, Matthaios Olma, Robert Grandl, Surajit Chaudhuri, and Bolin Ding. 2016. Quickr: Lazily approximating complex Ad-Hoc Queries in big data clusters. In *ACM SIGMOD Int'l. Conference on Management of Data (MOD)*. 631–646.

[109] Anil Kanduri, Antonio Miele, Amir M. Rahmani, Pasi Liljeberg, Cristiana Bolchini, and Nikil Dutt. 2018. Approximation-aware coordinated power/performance management for heterogeneous multi-cores. In *Design Automation Conference (DAC)*. 1–6.

[110] Seokwon Kang, Kyunghwan Choi, and Yongjun Park. 2020. PreScaler: An efficient system-aware precision scaling framework on heterogeneous systems. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization*. 280–292.

[111] Mustafa Karakoy, Orhan Kislal, Xulong Tang, Mahmut Taylan Kandemir, and Meenakshi Arunachalam. 2019. Architecture-aware approximate computing. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 3, 2 (2019), 1–24.

[112] Georgios Keramidas, Chrysa Kokkala, and Iakovos Stamoulis. 2015. Clumsy value cache: An approximate memoization technique for mobile GPU fragment shaders. In *Workshop on Approximate Computing (WAPCO)*. 1–6.

[113] Min Soo Kim, Alberto A Del Barrio, Leonardo Tavares Oliveira, Roman Hermida, and Nader Bagherzadeh. 2018. Efficient mitchell's approximate log multipliers for convolutional neural networks. *IEEE Trans. on Computers* 68, 5 (2018), 660–675.

[114] Sung Kim, Patrick Howe, Thierry Moreau, Armin Alaghi, Luis Ceze, and Visvesh Sathe. 2018. MATIC: Learning around errors for efficient low-voltage neural network accelerators. In *Design, Automation and Test in Europe (DATE)*. 1–6.

[115] Younghoon Kim, Swagath Venkataramani, Sanchari Sen, and Anand Raghunathan. 2021. Value similarity extensions for approximate computing in general-purpose processors. In *Design, Automation and Test in Europe (DATE)*. 481–486.

[116] Yongtae Kim, Yong Zhang, and Peng Li. 2013. An energy efficient approximate adder with carry skip for error resilient neuromorphic VLSI systems. In *Int'l. Conference on Computer-Aided Design (ICCAD)*. 130–137.

[117] Orhan Kislal and Mahmut T. Kandemir. 2018. Data access skipping for recursive partitioning methods. *Elsevier Computer Languages, Systems and Structures* 53 (2018), 143–162.

[118] Skanda Koppula, Lois Orosa, A Giray Yağlıkçı, Roknoddin Azizi, Taha Shahroodi, Konstantinos Kanellopoulos, and Onur Mutlu. 2019. EDEN: Enabling energy-efficient, high-performance deep neural network inference using approximate DRAM. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 166–181.

[119] Raghuraman Krishnamoorthi. 2018. Quantizing deep convolutional networks for efficient inference: A whitepaper. 1–36.

[120] Dhanya R. Krishnan, Do Le Quoc, Pramod Bhatotia, Christof Fetzer, and Rodrigo Rodrigues. 2016. IncApprox: A data analytics system for incremental approximate computing. In *Int'l. Conf.on World Wide Web*. 1133–1144.

[121] Animesh Kumar, Jan Rabaey, and Kannan Ramchandran. 2009. SRAM supply voltage scaling: A reliability perspective. In *Int'l. Symposium on Quality Electronic Design (ISQED)*. 782–787.

[122] Fadi J. Kurdahi, Ahmed Eltawil, Kang Yi, Stanley Cheng, and Amin Khajeh. 2010. Low-power multimedia system design by aggressive voltage scaling. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 18, 5 (2010), 852–856.

[123] Christos Kyrkou, George Plastiras, Theocharis Theocharides, Stylianos I Venieris, and Christos-Savvas Bouganis. 2018. DroNet: Efficient convolutional neural network detector for real-time UAV applications. In *Design, Automation and Test in Europe (DATE)*. 967–972.

[124] Ignacio Laguna, Paul C. Wood, Ranvijay Singh, and Saurabh Bagchi. 2019. GPUMixer: Performance-driven floating-point tuning for GPU scientific applications. In *ISC Int'l. Conference on High Performance Computing (HPC)*. 227–246.

[125] Michael O. Lam and Jeffrey K. Hollingsworth. 2018. Fine-grained floating-point precision analysis. *SAGE Int'l. Journal of High Performance Computing Applications* 32, 2 (2018), 231–245.

[126] Michael O. Lam, Jeffrey K. Hollingsworth, Bronis R. de Supinski, and Matthew P. Legendre. 2013. Automatically adapting programs for mixed-precision floating-point computation. In *ACM Int'l. Conference on Supercomputing (ICS)*. 369–378.

[127] Nikolay Laptev, Kai Zeng, and Carlo Zaniolo. 2012. Early accurate results for advanced analytics on MapReduce. *Proceedings of the VLDB Endowment* 5, 10 (2012), 1028–1039.

[128] Vadim Lebedev and Victor Lempitsky. 2016. Fast ConvNets using group-wise brain damage. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2554–2564.

[129] Chunho Lee, Miodrag Potkonjak, and William Henry Mangione-Smith. 1997. MediaBench: A tool for evaluating and synthesizing multimedia and communications systems. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 330–335.

[130] Seogoo Lee, Lizy K. John, and Andreas Gerstlauer. 2017. High-level synthesis of approximate hardware under joint precision and voltage scaling. In *Design, Automation and Test in Europe (DATE)*. 187–192.

[131] George Lentaris, George Chatzitsompanis, Vasileios Leon, Kiamal Pekmestzi, and Dimitrios Soudris. 2020. Combining arithmetic approximation techniques for improved CNN circuit design. In *IEEE Int'l. Conference on Electronics, Circuits and Systems (ICECS)*. 1–4.

[132] Vasileios Leon, Muhammad Abdullah Hanif, Giorgos Armeniakos, Xun Jiao, Muhammad Shafique, Kiamal Pekmestzi, and Dimitrios Soudris. 2025. Approximate computing survey, part I: Terminology and software and hardware approximation techniques. *Comput. Surveys* (2025), 1–35.

[133] Vasileios Leon, Georgios Makris, Sotirios Xydis, Kiamal Pekmestzi, and Dimitrios Soudris. 2022. MAx-DNN: Multi-level arithmetic approximation for energy-efficient DNN hardware accelerators. In *IEEE Latin America Symposium on Circuits and System (LASCAS)*. 1–4.

[134] Vasileios Leon, Theodora Paparouni, Evangelos Petrongonas, Dimitrios Soudris, and Kiamal Pekmestzi. 2021. Improving power of DSP and CNN hardware accelerators using approximate floating-point multipliers. *ACM Trans. on Embedded Computing Systems* 20, 5 (2021), 1–21.

[135] Vasileios Leon, Ioannis Stratakos, Giorgos Armeniakos, George Lentaris, and Dimitrios Soudris. 2021. ApproxQAM: High-order QAM demodulation circuits with approximate arithmetic. In *Int'l. Conference on Modern Circuits and Systems Technologies (MOCAST)*. 1–5.

[136] Vasileios Leon, Georgios Zervakis, Dimitrios Soudris, and Kiamal Pekmestzi. 2018. Approximate hybrid high radix encoding for energy-efficient inexact multipliers. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 26, 3 (2018), 421–430.

[137] Vasileios Leon, Georgios Zervakis, Sotirios Xydis, Dimitrios Soudris, and Kiamal Pekmestzi. 2018. Walking through the energy-error pareto frontier of approximate multipliers. *IEEE Micro* 38, 4 (2018), 40–49.

[138] Fei Li, Youyou Lu, Zhongjie Wu, and Jiwu Shu. 2019. ASCache: An approximate SSD cache for error-tolerant applications. In *Design Automation Conference (DAC)*.

[139] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. Pruning filters for efficient ConvNets. *International Conference on Learning Representations (ICLR'17)*. 1–13.

[140] Shikai Li, Sunghyun Park, and Scott Mahlke. 2018. Sculptor: Flexible approximation with selective dynamic loop perforation. In *ACM Int'l. Conference on Supercomputing (ICS)*. 341–351.

[141] Yingyan Lin, Charbel Sakr, Yongjune Kim, and Naresh Shanbhag. 2017. PredictiveNet: an energy-efficient convolutional neural network via zero prediction. In *IEEE Int'l. Symposium on Circuits and Systems (ISCAS)*. 1–4.

[142] Michael D. Linderman, Matthew Ho, David L. Dill, Teresa H. Meng, and Garry P. Nolan. 2010. Towards program optimization through automated analysis of numerical precision. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization (CGO)*. 230–237.

[143] Gai Liu and Zhiru Zhang. 2017. Statistically certified approximate logic synthesis. In *Int'l. Conference on Computer-Aided Design (ICCAD)*. 344–351.

[144] Ren-Shuo Liu, Chia-Lin Yang, and Wei Wu. 2012. Optimizing NAND flash-based SSDs via retention relaxation. In *USENIX Conference on File and Storage Technologies (FAST)*. 1–14.

[145] Song Liu, Karthik Pattabiraman, Thomas Moscibroda, and Benjamin G. Zorn. 2011. Flikker: Saving DRAM refresh-power through critical data partitioning. In *ACM Int'l. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 213–224.

[146] Weiqiang Liu, Jing Li, Tao Xu, Chenghua Wang, Paolo Montuschi, and Fabrizio Lombardi. 2018. Combining restoring array and logarithmic dividers into an approximate hybrid design. In *IEEE Symposium on Computer Arithmetic (ARITH)*. 92–98.

[147] Weiqiang Liu, Liangyu Qian, Chenghua Wang, Honglan Jiang, Jie Han, and Fabrizio Lombardi. 2017. Design of approximate radix-4 Booth multipliers for error-tolerant computing. *IEEE Trans. on Computers* 66, 8 (2017), 1435–1441.

[148] Weiqiang Liu, Jiahua Xu, Danye Wang, Chenghua Wang, Paolo Montuschi, and Fabrizio Lombardi. 2018. Design and evaluation of approximate logarithmic multipliers for low power error-tolerant applications. *IEEE Trans. on Circuits and Systems I: Regular Papers* 65, 9 (2018), 2856–2868.

[149] Yang Liu, Tong Zhang, and Keshab K. Parhi. 2010. Computation error analysis in digital signal processing systems with overscaled supply voltage. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 18, 4 (2010), 517–526.

[150] Zhenhong Liu, Amir Yazdanbakhsh, Dong Kai Wang, Hadi Esmaeilzadeh, and Nam Sung Kim. 2019. AxMemo: Hardware-compiler co-design for approximate code memoization. In *ACM/IEEE Int'l. Symposium on Computer Architecture (ISCA)*. 685–697.

[151] Wei Lou, Lei Xun, Amin Sabet, Jia Bi, Jonathon Hare, and Geoff V Merrett. 2021. Dynamic-OFA: Runtime DNN architecture switching for performance scaling on heterogeneous embedded platforms. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 3110–3118.

[152] Dongning Ma, Rahul Thapa, Xingjian Wang, Xun Jiao, and Cong Hao. 2021. Workload-aware approximate computing configuration. In *Design, Automation and Test in Europe (DATE)*. 920–925.

[153] Dongning Ma, Xinqiao Zhang, Ke Huang, Yu Jiang, Wanli Chang, and Xun Jiao. 2022. DEVoT: Dynamic delay modeling of functional units under voltage and temperature variations. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 41, 4 (2022), 827–839.

[154] Divya Mahajan, Amir Yazdanbaksh, Jongse Park, Bradley Thwaites, and Hadi Esmaeilzadeh. 2016. Towards statistical guarantees in controlling quality tradeoffs for approximate acceleration. In *ACM/IEEE Annual Int'l. Symposium on Computer Architecture (ISCA)*. 66–77.

[155] K. Manikantta Reddy, M. H. Vasantha, Y. B. Nithin Kumar, and Devesh Dwivedi. 2020. Design of approximate Booth squarer for error-tolerant computing. *IEEE Trans. on Very Large Scale Integration Systems* 28, 5 (2020), 1230–1241.

[156] Jiayuan Meng, Srimat Chakradhar, and Anand Raghunathan. 2009. Best-effort parallel execution framework for recognition and mining applications. In *IEEE Int'l. Symposium on Parallel Distributed Processing (IPDPS)*. 1–12.

[157] Jiayuan Mengt, Anand Raghunathan, Srimat Chakradhar, and Surendra Byna. 2010. Exploiting the forgiving nature of applications for scalable parallel execution. In *IEEE Int'l. Symposium on Parallel Distributed Processing*. 1–12.

[158] Harshitha Menon, Michael O. Lam, Daniel Osei-Kuffuor, Markus Schordan, Scott Lloyd, Kathryn Mohror, and Jeffrey Hittinger. 2018. ADAPT: Algorithmic differentiation applied to floating-point precision tuning. In *ACM/IEEE SC, Int'l. Conference for High Performance Computing, Networking, Storage and Analysis*. 614–626.

[159] Joshua San Miguel, Jorge Albericio, Natalie Enright Jerger, and Aamer Jaleel. 2016. The bunker cache for spatio-value approximation. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 1–12.

[160] Joshua San Miguel, Jorge Albericio, Andreas Moshovos, and Natalie Enright Jerger. 2015. Doppelgänger: A cache for approximate computing. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 50–61.

[161] Joshua San Miguel, Mario Badr, and Natalie Enright Jerger. 2014. Load value approximation. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 127–139.

[162] Sasa Misailovic, Michael Carbin, Sara Achour, Zichao Qi, and Martin C. Rinard. 2014. Chisel: Reliability- and accuracy-aware optimization of approximate computational kernels. In *ACM SIGPLAN Int'l. Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 309–328.

[163] Sasa Misailovic, Deokhwan Kim, and Martin C. Rinard. 2013. Parallelizing sequential programs with statistical accuracy tests. *ACM Trans. on Embedded Computing Systems* 12, 2s (2013), 1–26.

[164] Sasa Misailovic, Stelios Sidiroglou, Henry Hoffmann, and Martin C. Rinard. 2010. Quality of service profiling. In *ACM/IEEE Int'l. Conference on Software Engineering (ICSE)*, Vol. 1. 25–34.

[165] Sasa Misailovic, Stelios Sidiroglou, and Martin C. Rinard. 2012. Dancing with uncertainty. In *ACM Workshop on Relaxing Synchronization for Multicore and Manycore Scalability (RACES)*. 51–60.

[166] Asit K. Mishra, Rajkishore Barik, and Somnath Paul. 2014. iACT: A software-hardware framework for understanding the scope of approximate computing. In *Workshop on Approximate Computing Across the System Stack*. 1–6.

[167] Subrata Mitra, Manish K. Gupta, Sasa Misailovic, and Saurabh Bagchi. 2017. Phase-aware optimization in approximate computing. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization (CGO)*. 185–196.

[168] Sparsh Mittal. 2016. A survey of techniques for approximate computing. *Comput. Surveys* 48, 4 (2016), 1–33.

[169] Sparsh Mittal and Jeffrey S. Vetter. 2015. A survey of CPU-GPU heterogeneous computing techniques. *Comput. Surveys* 47, 4 (2015), 1–35.

[170] Debabrata Mohapatra, Vinay K. Chippa, Anand Raghunathan, and Kaushik Roy. 2011. Design of voltage-scalable meta-functions for approximate computing. In *Design, Automation and Test in Europe (DATE)*. 1–6.

[171] Amir Momeni, Jie Han, Paolo Montuschi, and Fabrizio Lombardi. 2015. Design and analysis of approximate compressors for multiplication. *IEEE Trans. on Computers* 64, 4 (2015), 984–994.

[172] Thierry Moreau, Joshua San Miguel, Mark Wyse, James Bornholt, Armin Alaghi, Luis Ceze, Natalie Enright Jerger, and Adrian Sampson. 2018. A taxonomy of general purpose approximate computing techniques. *IEEE Embedded Systems Letters* 10, 1 (2018), 2–5.

[173] Thierry Moreau, Mark Wyse, Jacob Nelson, Adrian Sampson, Hadi Esmaeilzadeh, Luis Ceze, and Mark Oskin. 2015. SNNAP: Approximate computing on programmable SoCs via neural acceleration. In *IEEE Int'l. Symposium on High Performance Computer Architecture (HPCA)*. 603–614.

[174] Vojtech Mrazek, Radek Hrbacek, Zdenek Vasicek, and Lukas Sekanina. 2017. EvoApprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods. In *Design, Automation and Test in Europe (DATE)*. 258–261.

[175] Vojtech Mrazek, Syed Shakib Sarwar, Lukas Sekanina, Zdenek Vasicek, and Kaushik Roy. 2016. Design of power-efficient approximate multipliers for approximate artificial neural networks. In *IEEE/ACM Int'l. Conference on Computer-Aided Design (ICCAD)*. 1–7.

[176] Vojtech Mrazek, Lukas Sekanina, and Zdenek Vasicek. 2020. Libraries of approximate circuits: Automated design and application in CNN accelerators. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 10, 4 (2020), 406–418.

[177] Vojtech Mrazek, Zdenek Vasícek, Lukás Sekanina, Muhammad Abdullah Hanif, and Muhammad Shafique. 2019. ALWANN: Automatic layer-wise approximation of deep neural network accelerators without retraining. In *IEEE/ACM Int'l. Conference on Computer-Aided Design (ICCAD)*. 1–8.

[178] Muhammad Husnain Mubarik, Dennis D. Weller, Nathaniel Bleier, Matthew Tomei, Jasmin Aghassi-Hagmann, Mehdi B. Tahoori, and Rakesh Kumar. 2020. Printed machine learning classifiers. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 73–87.

[179] Ramanathan Narayanan, Berkin Ozisikyilmaz, Joseph Zambreno, Gokhan Memik, and Alok Choudhary. 2006. MineBench: A benchmark suite for data mining workloads. In *IEEE Int'l. Symposium on Workload Characterization (IISWC)*. 182–188.

[180] Geneviève Ndour, Tiago Trevisan Jost, Anca Molnos, Yves Durand, and Arnaud Tisserand. 2019. Evaluation of variable bit-width units in a RISC-V processor for approximate computing. In *ACM Int'l. Conference on Computing Frontiers*. 344–349.

[181] Kumud Nepal, Soheil Hashemi, Hokchhay Tann, R. Iris Bahar, and Sherief Reda. 2019. Automated high-level generation of low-power approximate computing circuits. *IEEE Trans. on Emerging Topics in Computing* 7, 1 (2019), 18–30.

[182] Kumud Nepal, Yueting Li, R. Iris Bahar, and Sherief Reda. 2014. ABACUS: A technique for automated behavioral synthesis of approximate computing circuits. In *Design, Automation and Test in Europe (DATE)*. 1–6.

[183] Gianmarco Ottavi, Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Luca Benini, and Davide Rossi. 2020. A mixed-precision RISC-V processor for extreme-edge DNN inference. In *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 512–517.

[184] Priyadarshini Panda, Abhronil Sengupta, and Kaushik Roy. 2017. Energy-efficient and improved image recognition with conditional deep learning. *ACM Journal on Emerging Technologies in Computing Systems* 13, 3 (2017), 1–21.

[185] Priyadarshini Panda, Abhronil Sengupta, Syed Shakib Sarwar, Gopalakrishnan Srinivasan, Swagath Venkataramani, Anand Raghunathan, and Kaushik Roy. 2016. Cross-layer approximations for neuromorphic computing: From devices to circuits and systems. In *Design Automation Conference (DAC)*. 1–6.

[186] Priyadarshini Panda, Swagath Venkataramani, Abhronil Sengupta, Anand Raghunathan, and Kaushik Roy. 2017. Energy-efficient object detection using semantic decomposition. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 25, 9 (2017), 2673–2677.

[187] Pramesh Pandey, Prabal Basu, Koushik Chakraborty, and Sanghamitra Roy. 2019. GreenTPU: Improving timing error resilience of a near-threshold tensor processing unit. In *Design Automation Conference (DAC)*. 1–6.

[188] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Brucek Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 27–40.

[189] Jongse Park, Hadi Esmaeilzadeh, Xin Zhang, Mayur Naik, and William Harris. 2015. FlexJava: Language support for safe and modular approximate programming. In *ACM SIGSOFT Symposium and European Conference on Foundations of Software Engineering (FSE)*. 745–757.

[190] Jongse Park, Xin Zhang, Kangqi Ni, Hadi Esmaeilzadeh, and Mayur Naik. 2014. ExpAX: A framework for automating approximate programming. *Georgia Institute of Technology Technical Report* GT-CS-14-05 (2014), 1–17. https://repository.gatech.edu/entities/publication/81a9735b-874d-4524-9555-cf935aaa0c9f

[191] Jun-Seok Park, Jun-Woo Jang, Heonsoo Lee, Dongwoo Lee, Sehwan Lee, and Hanwoong Jung. 2021. A 6K-MAC feature-map-sparsity-aware neural processing unit in 5nm flagship mobile SoC. In *IEEE Int'l. Solid-State Circuits Conference (ISSCC)*, Vol. 64. 152–154.

[192] Yongjoo Park, Jingyi Qing, Xiaoyang Shen, and Barzan Mozafari. 2019. BlinkML: Efficient maximum likelihood estimation with probabilistic guarantees. In *ACM SIGMOD Int'l. Conference on Management of Data (MOD)*. 1135–1152.

[193] Masoud Pashaeifar, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2018. Approximate reverse carry propagate adder for energy-efficient DSP applications. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 26, 11 (2018), 2530–2541.

[194] Ratko Pilipović, Patricio Bulić, and Uroš Lotrič. 2021. A two-stage operand trimming approximate logarithmic multiplier. *IEEE Trans. on Circuits and Systems I: Regular Papers* 68, 6 (2021), 2535–2545.

[195] George Plastiras, Christos Kyrkou, and Theocharis Theocharides. 2018. Efficient ConvNet-based object detection for unmanned aerial vehicles by selective tile processing. In *Int'l. Conf. on Distributed Smart Cameras (ICDSC)*. 1–6.

[196] Louis-Noel Pouchet. 2015. *PolyBench*. Retrieved from http://sourceforge.net/projects/polybench/

[197] Roldan Pozo and Bruce Miller. 2004. *SciMark 2.0*. Retrieved from http://math.nist.gov/scimark2/

[198] Do Le Quoc, Martin Beck, Pramod Bhatotia, Ruichuan Chen, Christof Fetzer, and Thorsten Strufe. 2017. PrivApprox: Privacy-preserving stream analytics. In *USENIX Annual Technical Conference (ATC)*. 659–672.

[199] Do Le Quoc, Ruichuan Chen, Pramod Bhatotia, Christof Fetzer, Volker Hilt, and Thorsten Strufe. 2017. StreamApprox: Approximate computing for stream analytics. In *ACM/IFIP/USENIX Int'l. Middleware Conference*. 185–197.

[200] Rengarajan Ragavan, Cedric Killian, and Olivier Sentieys. 2016. Adaptive overclocking and error correction based on dynamic speculation window. In *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 325–330.

[201] Arnab Raha and Vijay Raghunathan. 2017. qLUT: Input-aware quantized table lookup for energy-efficient approximate accelerators. *ACM Trans. on Embedded Computing Systems* 16, 5s (2017), 130:1–130:23.

[202] Arnab Raha and Vijay Raghunathan. 2017. Towards full-system energy-accuracy tradeoffs: A case study of an approximate smart camera system. In *Design Automation Conference (DAC)*. 1–6.

[203] Arnab Raha and Vijay Raghunathan. 2018. Approximating beyond the processor: Exploring full-system energy-accuracy tradeoffs in a smart camera system. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 26, 12 (2018), 2884–2897.

[204] Arnab Raha, Soubhagya Sutar, Hrishikesh Jayakumar, and Vijay Raghunathan. 2017. Quality configurable approximate DRAM. *IEEE Trans. on Computers* 66, 7 (2017), 1172–1187.

[205] Arnab Raha, Swagath Venkataramani, Vijay Raghunathan, and Anand Raghunathan. 2015. Quality configurable reduce-and-rank for energy efficient approximate computing. In *Design, Automation and Test in Europe (DATE)*. 665–670.

[206] Abbas Rahimi, Luca Benini, and Rajesh K. Gupta. 2013. Spatial memoization: Concurrent instruction reuse to correct timing errors in SIMD architectures. *IEEE Trans. on Circuits and Systems II: Express Briefs* 60, 12 (2013), 847–851.

[207] Shankar Ganesh Ramasubramanian, Swagath Venkataramani, Adithya Parandhaman, and Anand Raghunathan. 2013. Relax-and-retime: A methodology for energy-efficient recovery based design. In *Design Automation Conference (DAC)*. 1–6.

[208] Ashish Ranjan, Arnab Raha, Vijay Raghunathan, and Anand Raghunathan. 2020. Approximate memory compression. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 28, 4 (2020), 980–991.

[209] Ashish Ranjan, Arnab Raha, Swagath Venkataramani, Kaushik Roy, and Anand Raghunathan. 2014. ASLAN: Synthesis of approximate sequential circuits. In *Design, Automation and Test in Europe (DATE)*. 1–6.

[210] Ashish Ranjan, Swagath Venkataramani, Xuanyao Fong, Kaushik Roy, and Anand Raghunathan. 2015. Approximate storage for energy efficient spintronic memories. In *Design Automation Conference (DAC)*. 1–6.

[211] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *European Conference on Computer Vision (ECCV)*. 525–542.

[212] Lakshminarayanan Renganarayana, Vijayalakshmi Srinivasan, Ravi Nair, and Daniel Prener. 2012. Programming with relaxed synchronization. In *ACM Workshop on Relaxing Synchronization for Multicore and Manycore Scalability (RACES)*. 41–50.

[213] Pedro Reviriego, Shanshan Liu, Otmar Ertl, Farzad Niknia, and Fabrizio Lombardi. 2022. Computing the similarity estimate using approximate memory. *IEEE Trans. on Emerging Topics in Computing* 10, 3 (2022), 1593–1604.

[214] Martin C. Rinard. 2006. Probabilistic accuracy bounds for fault-tolerant computations that discard tasks. In *ACM Int'l. Conference on Supercomputing (ICS)*. 324–334.

[215] Martin C. Rinard. 2007. Using early phase termination to eliminate load imbalances at barrier synchronization points. In *ACM SIGPLAN Int'l. Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 369–386.

[216] Cindy Rubio-González, Cuong Nguyen, Benjamin Mehne, Koushik Sen, James Demmel, William Kahan, Costin Iancu, Wim Lavrijsen, David H. Bailey, and David Hough. 2016. Floating-point precision tuning using Blame Analysis. In *IEEE/ACM Int'l. Conference on Software Engineering (ICSE)*. 1074–1085.

[217] Cindy Rubio-González, Cuong Nguyen, Hong Diep Nguyen, James Demmel, William Kahan, Koushik Sen, David H. Bailey, Costin Iancu, and David Hough. 2013. Precimonious: Tuning assistant for floating-point precision. In *SC13: Int'l. Conference on High Performance Computing, Networking, Storage and Analysis*. 1–12.

[218] Hassaan Saadat, Haseeb Bokhari, and Sri Parameswaran. 2018. Minimally biased multipliers for approximate integer and floating-point multiplication. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2623–2635.

[219] Hassaan Saadat, Haris Javaid, Aleksandar Ignjatovic, and Sri Parameswaran. 2020. REALM: Reduced-error approximate log-based integer multiplier. In *Design, Automation and Test in Europe (DATE)*. 1366–1371.

[220] Hassaan Saadat, Haris Javaid, and Sri Parameswaran. 2019. Approximate integer and floating-point dividers with near-zero error bias. In *Design Automation Conference (DAC)*. 1–6.

[221] Farnaz Sabetzadeh, Mohammad Hossein Moaiyeri, and Mohammad Ahmadinejad. 2019. A majority-based imprecise multiplier for ultra-efficient approximate image multiplication. *IEEE Trans. on Circuits and Systems I: Regular Papers* 66, 11 (2019), 4200–4208.

[222] Mastooreh Salajegheh, Yue Wang, Kevin Fu, Anxiao Jiang, and Erik Learned-Miller. 2011. Exploiting half-wits: Smarter storage for low-power devices. In *USENIX Conference on File and Storage Technologies (FAST)*. 1–14.

[223] Mehrzad Samadi, Davoud Anoushe Jamshidi, Janghaeng Lee, and Scott Mahlke. 2014. Paraprox: Pattern-based approximation for data parallel applications. In *ACM Int'l. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 35–50.

[224] Mehrzad Samadi, Janghaeng Lee, D. Anoushe Jamshidi, Amir Hormati, and Scott Mahlke. 2013. SAGE: Self-tuning approximation for graphics engines. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 13–24.

[225] Felipe Sampaio, Muhammad Shafique, Bruno Zatt, Sergio Bampi, and Jörg Henkel. 2015. Approximation-aware multi-level cells STT-RAM cache architecture. In *Int'l. Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*. 79–88.

[226] Adrian Sampson, André Baixo, Benjamin Ransford, Thierry Moreau, Joshua Yip, Luis Ceze, and Mark Oskin. 2015. ACCEPT: A programmer-guided compiler framework for practical approximate computing. *University of Washington Technical Report* UW-CSE-15-01-01 (2015), 1–14. https://dada.cs.washington.edu/research/tr/2015/01/UW-CSE-15-01-01.pdf)

[227] Adrian Sampson, Werner Dietl, Emily Fortuna, Danushen Gnanapragasam, Luis Ceze, and Dan Grossman. 2011. EnerJ: Approximate data types for safe and general low-power computation. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 164–174.

[228] Adrian Sampson, Jacob Nelson, Karin Strauss, and Luis Ceze. 2014. Approximate storage in solid-state memories. *ACM Trans. on Computer Systems* 32, 3 (2014), 1–23.

[229] John Sartori and Rakesh Kumar. 2011. Architecting processors to allow voltage/reliability tradeoffs. In *Int'l. Conference on Compilers, Architectures and Synthesis for Embedded Systems (CASES)*. 115–124.

[230] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2014. Stochastic optimization of floating-point programs with tunable precision. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 53–64.

[231] Jeremy Schlachter, Vincent Camus, Krishna V. Palem, and Christian Enz. 2017. Design and applications of approximate circuits by gate-level pruning. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 25, 5 (2017), 1694–1702.

[232] Muhammad Shafique, Rehan Hafiz, Semeen Rehman, Walaa El-Harouni, and Jörg Henkel. 2016. Cross-layer approximate computing: From logic to architectures. In *Design Automation Conference (DAC)*. 1–6.

[233] Kan Shi, David Boland, and George A. Constantinides. 2013. Accuracy-performance tradeoffs on an FPGA through overclocking. In *IEEE Int'l. Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 29–36.

[234] Kan Shi, David Boland, Edward Stott, Samuel Bayliss, and George A. Constantinides. 2014. Datapath synthesis for overclocking: Online arithmetic for latency-accuracy trade-offs. In *Design Automation Conference (DAC)*. 1–6.

[235] Qingchuan Shi, Henry Hoffmann, and Omer Khan. 2015. A cross-layer multicore architecture to tradeoff program accuracy and resilience overheads. *IEEE Computer Architecture Letters* 14, 2 (2015), 85–89.

[236] Dongyeob Shin, Wonseok Choi, Jongsun Park, and Swaroop Ghosh. 2019. Sensitivity-based error resilient techniques with heterogeneous multiply–accumulate unit for voltage scalable deep neural network accelerators. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 3 (2019), 520–531.

[237] Stelios Sidiroglou-Douskos, Sasa Misailovic, Henry Hoffmann, and Martin C. Rinard. 2011. Managing performance vs. accuracy trade-offs with loop perforation. In *ACM SIGSOFT Symposium and European Conference on Foundations of Software Engineering (FSE)*. 124–134.

[238] Lorenzo Sonnino, Shaswot Shresthamali, Yuan He, and Masaaki Kondo. 2024. DAISM: Digital approximate In-SRAM multiplier-based accelerator for DNN training and inference. *Design, Automation and Test in Europe Conference (DATE'24)*. 1–6.

[239] Jacob Sorber, Alexander Kostadinov, Matthew Garber, Matthew Brennan, Mark D. Corner, and Emery D. Berger. 2007. Eon: A language and runtime system for perpetual systems. In *ACM Int'l. Conference on Embedded Networked Sensor Systems (SenSys)*. 161–174.

[240] The Open Source. 2023. History—RISC-V international. (2023). Retrieved from https://riscv.org/about/history/
[241] Ourania Spantidi, Georgios Zervakis, Iraklis Anagnostopoulos, Hussam Amrouch, and Jörg Henkel. 2021. Positive/negative approximate multipliers for DNN accelerators. In *IEEE/ACM Int'l. Conference On Computer Aided Design (ICCAD)*. 1–9.
[242] Jaswanth Sreeram and Santosh Pande. 2010. Exploiting approximate value locality for data synchronization on multi-core processors. In *IEEE Int'l. Symposium on Workload Characterization (IISWC)*. 1–10.
[243] Phillip Stanley-Marbell, Armin Alaghi, Michael Carbin, Eva Darulova, Lara Dolecek, Andreas Gerstlauer, Ghayoor Gillani, Djordje Jevdjic, Thierry Moreau, Mattia Cacciotti, Alexandros Daglis, Natalie Enright Jerger, Babak Falsafi, Sasa Misailovic, Adrian Sampson, and Damien Zufferey. 2020. Exploiting errors for efficiency: A survey from circuits to applications. *Comput. Surveys* 53, 3 (2020), 1–39.
[244] Greg Stitt and David Campbell. 2020. PANDORA: An architecture-independent parallelizing approximation-discovery framework. *ACM Trans. on Embedded Computing Systems* 19, 5 (2020), 1–17.
[245] Antonio Giuseppe Maria Strollo, Ettore Napoli, Davide De Caro, Nicola Petra, and Gennaro Di Meo. 2020. Comparison and extension of approximate 4-2 compressors for low-power approximate multipliers. *IEEE Trans. on Circuits and Systems I: Regular Papers* 67, 9 (2020), 3021–3034.
[246] Yang Sui, Miao Yin, Yi Xie, Huy Phan, Saman Aliari Zonouz, and Bo Yuan. 2021. CHIP: CHannel independence-based pruning for compact neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 24604–24616.
[247] Mahdi Taheri, Mohammad Riazati, Mohammad Hasan Ahmadilivani, Maksim Jenihhin, Masoud Daneshtalab, Jaan Raik, Mikael Sjodin, and Bjorn Lisper. 2023. DeepAxe: A framework for exploration of approximation and reliability trade-offs in DNN accelerators. *International Symposium on Quality Electronic Design (ISQED'23)*. 1–8.
[248] Cheng Tan, Thannirmalai Somu Muthukaruppan, Tulika Mitra, and Ju Lei. 2015. Approximation-aware scheduling on heterogeneous multi-core architectures. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*. 618–623.
[249] Chong Min John Tan and Mehul Motani. 2020. DropNet: Reducing neural network complexity via iterative pruning. In *Int'l. Conference on Machine Learning (ICML)*. 9356–9366.
[250] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Int'l. Conference on Machine Learning (ICML)*. 6105–6114.
[251] Hokchhay Tann, Soheil Hashemi, R Iris Bahar, and Sherief Reda. 2016. Runtime configurable deep neural networks for energy-accuracy trade-off. In *Conference on Hardware/Software Codesign and System Synthesis*. 1–10.
[252] Mohammad Taghi Teimoori, Muhammad Abdullah Hanif, Alireza Ejlali, and Muhammad Shafique. 2018. AdAM: Adaptive approximation management for the non-volatile memory hierarchies. In *Design, Automation and Test in Europe Conference (DATE)*. 785–790.
[253] Ye Tian, Qian Zhang, Ting Wang, Feng Yuan, and Qiang Xu. 2015. ApproxMA: Approximate memory access for dynamic precision scaling. In *Great Lakes Symposium on VLSI (GLSVLSI)*. 337–342.
[254] David Tolpin, Jan-Willem van de Meent, Hongseok Yang, and Frank Wood. 2016. Design and implementation of probabilistic programming language anglican. In *Symposium on Implementation and Application of Functional Programming Languages (IFL)*. 1–12.
[255] Hung-Wei Tseng, Laura M. Grupp, and Steven Swanson. 2013. Underpowering NAND flash: Profits and perils. In *Design Automation Conference (DAC)*. 1–6.
[256] Georgios Tziantzioulis, Nikos Hardavellas, and Simone Campanoni. 2018. Temporal approximate function memoization. *IEEE Micro* 38, 4 (2018), 60–70.
[257] Salim Ullah, Sanjeev Sripadraj Murthy, and Akash Kumar. 2018. SMApproxLib: Library of FPGA-based approximate multipliers. In *Design Automation Conference (DAC)*. 1–6.
[258] Shaghayegh Vahdat, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2019. TOSAM: An energy-efficient truncation- and rounding-based scalable approximate multiplier. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 27, 5 (2019), 1161–1173.
[259] Shaghayegh Vahdat, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Zainalabedin Navabi. 2017. TruncApp: A truncation-based approximate divider for energy efficient DSP applications. In *Design, Automation and Test in Europe (DATE)*. 1635–1638.
[260] Zdenek Vasicek, Vojtech Mrazek, and Lukas Sekanina. 2019. Automated circuit approximation method driven by data distribution. In *Design, Automation and Test in Europe (DATE)*. 96–101.
[261] Zdenek Vasicek and Lukas Sekanina. 2015. Evolutionary approach to approximate digital circuits design. *IEEE Trans. on Evolutionary Computation* 19, 3 (2015), 432–444.
[262] Vassilis Vassiliadis, Konstantinos Parasyris, Charalambos Chalios, Christos D. Antonopoulos, Spyros Lalis, Nikolaos Bellas, Hans Vandierendonck, and Dimitrios S. Nikolopoulos. 2015. A programming model and runtime system for significance-aware energy-efficient computing. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 275–276.

[263] Vassilis Vassiliadis, Jan Riehme, Jens Deussen, Konstantinos Parasyris, Christos D. Antonopoulos, Nikolaos Bellas, Spyros Lalis, and Uwe Naumann. 2016. Towards automatic significance analysis for approximate computing. In *IEEE/ACM Int'l. Symposium on Code Generation and Optimization (CGO)*. 182–193.

[264] Suganthi Venkatachalam, Elizabeth Adams, Hyuk Jae Lee, and Seok-Bum Ko. 2019. Design and analysis of area and power efficient approximate Booth multipliers. *IEEE Trans. on Computers* 68, 11 (2019), 1697–1703.

[265] Swagath Venkataramani, Xiao Sun, Naigang Wang, Chia-Yu Chen, Jungwook Choi, Mingu Kang, Ankur Agarwal, Jinwook Oh, Shubham Jain, Tina Babinsky, Nianzheng Cao, Thomas Fox, Bruce Fleischer, George Gristede, Michael Guillorn, Howard Haynie, Hiroshi Inoue, Kazuaki Ishizaki, Michael Klaiber, Shih-Hsien Lo, Gary Maier, Silvia Mueller, Michael Scheuermann, Eri Ogawa, Marcel Schaal, Mauricio Serrano, Joel Silberman, Christos Vezyrtzis, Wei Wang, Fanchieh Yee, Jintao Zhang, Matthew Ziegler, Ching Zhou, Moriyoshi Ohara, Pong-Fei Lu, Brian Curran, Sunil Shukla, Vijayalakshmi Srinivasan, Leland Chang, and Kailash Gopalakrishnan. 2020. Efficient AI system design with cross-layer approximate computing. *Proc. IEEE* 108, 12 (2020), 2232–2250.

[266] Swagath Venkataramani, Victor Bahl, Xian-Sheng Hua, Jie Liu, Jin Li, Matthai Phillipose, Bodhi Priyantha, and Mohammed Shoaib. 2015. SAPPHIRE: An always-on context-aware computer vision system for portable devices. In *Design, Automation and Test in Europe (DATE)*. 1491–1496.

[267] Swagath Venkataramani, Srimat T. Chakradhar, Kaushik Roy, and Anand Raghunathan. 2015. Approximate computing and the quest for computing efficiency. In *Design Automation Conference (DAC)*. 1–6.

[268] Swagath Venkataramani, Anand Raghunathan, Jie Liu, and Mohammed Shoaib. 2015. Scalable-effort classifiers for energy-efficient machine learning. In *Design Automation Conference (DAC)*. 1–6.

[269] Swagath Venkataramani, Ashish Ranjan, Kaushik Roy, and Anand Raghunathan. 2014. AxNN: Energy-efficient neuromorphic systems using approximate computing. In *ACM/IEEE Int'l. Symposium on Low Power Electronics and Design (ISLPED)*. 27–32.

[270] Swagath Venkataramani, Kaushik Roy, and Anand Raghunathan. 2013. Substitute-and-Simplify: A unified design paradigm for approximate and quality configurable circuits. In *Design, Automation and Test in Europe (DATE)*. 1367–1372.

[271] Swagath Venkataramani, Amit Sabne, Vivek Kozhikkottu, Kaushik Roy, and Anand Raghunathan. 2012. SALSA: Systematic logic synthesis of approximate circuits. In *Design Automation Conference (DAC)*. 796–801.

[272] Huan Wang, Can Qin, Yulun Zhang, and Yun Fu. 2021. Neural pruning via growing regularization. *International Conference on Learning Representations (ICLR'21)*. 1–16.

[273] Jing Wang, Xin Fu, Xu Wang, Shubo Liu, Lan Gao, and Weigong Zhang. 2020. Enabling energy-efficient and reliable neural network via neuron-level voltage scaling. *IEEE Trans. on Computers* 69, 10 (2020), 1460–1473.

[274] Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. 2019. HAQ: Hardware-aware automated quantization with mixed precision. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 8612–8620.

[275] Ying Wang, Jiachao Deng, Yuntan Fang, Huawei Li, and Xiaowei Li. 2017. Resilience-aware frequency tuning for neural-network-based approximate computing chips. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 25, 10 (2017), 2736–2748.

[276] Haroon Waris, Chenghua Wang, and Weiqiang Liu. 2020. Hybrid low radix encoding-based approximate Booth multipliers. *IEEE Trans. on Circuits and Systems II: Express Briefs* 67, 12 (2020), 3367–3371.

[277] Haroon Waris, Chenghua Wang, Weiqiang Liu, and Fabrizio Lombardi. 2021. AxBMs: Approximate radix-8 Booth multipliers for high-performance FPGA-based accelerators. *IEEE Trans. on Circuits and Systems II: Express Briefs* 68, 5 (2021), 1566–1570.

[278] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. 2016. Learning structured sparsity in deep neural networks. *Advances in Neural Information Processing Systems* 29 (2016), 1–10.

[279] Zhenyu Wen, Do Le Quoc, Pramod Bhatotia, Ruichuan Chen, and Myungjin Lee. 2018. ApproxIoT: Approximate analytics for edge computing. In *IEEE Int'l. Conference on Distributed Computing Systems (ICDCS)*. 411–421.

[280] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. 1995. The SPLASH-2 programs: Characterization and methodological considerations. In *ACM/IEEE Annual Int'l. Symposium on Computer Architecture (ISCA)*. 24–36.

[281] Qiang Xu, Todd Mytkowicz, and Nam Sung Kim. 2016. Approximate computing: A survey. *IEEE Design and Test* 33, 1 (2016), 8–22.

[282] Amir Yazdanbakhsh, Divya Mahajan, Bradley Thwaites, Jongse Park, Anandhavel Nagendrakumar, and Sindhuja Sethuraman. 2015. Axilog: Language support for approximate hardware design. In *Design, Automation and Test in Europe (DATE)*. 812–817.

[283] Amir Yazdanbakhsh, Divya Mahajan, Hadi Esmaeilzadeh, and Pejman Lotfi-Kamran. 2017. AxBench: A multiplatform benchmark suite for approximate computing. *IEEE Design and Test* 34, 2 (2017), 60–68.

[284] Amir Yazdanbakhsh, Divya Mahajan, Pejman Lotfi-Kamran, and Hadi Esmaeilzadeh. 2016. *AxBench: A Benchmark Suite for Approximate Computing Across the System Stack*. Technical Report. Georgia Institute of Technology.

[285] Amir Yazdanbakhsh, Jongse Park, Hardik Sharma, Pejman Lotfi-Kamran, and Hadi Esmaeilzadeh. 2015. Neural acceleration for GPU throughput processors. In *IEEE/ACM Int'l. Symposium on Microarchitecture (MICRO)*. 482–493.

[286] Rong Ye, Ting Wang, Feng Yuan, Rakesh Kumar, and Qiang Xu. 2013. On reconfiguration-oriented approximate adder design and its application. In *Int'l. Conference on Computer-Aided Design (ICCAD)*. 48–54.

[287] Serif Yesil, Ismail Akturk, and Ulya R. Karpuzcu. 2018. Toward dynamic precision scaling. *IEEE Micro* 38, 4 (2018), 30–39.

[288] Jiecao Yu, Andrew Lukefahr, David Palframan, Ganesh Dasika, Reetuparna Das, and Scott Mahlke. 2017. Scalpel: Customizing DNN pruning to the underlying hardware parallelism. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 548–560.

[289] Edouard Yvinec, Arnaud Dapogny, Matthieu Cord, and Kevin Bailly. 2021. RED: Looking for redundancies for data-free structured compression of deep neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 20863–20873.

[290] Edouard Yvinec, Arnaud Dapogny, Matthieu Cord, and Kevin Bailly. 2022. RED++: Data-free pruning of deep neural networks via input splitting and output merging. *IEEE Trans. on Pattern Analysis and Machine Intelligence* 45, 3 (2022), 3664–3676.

[291] Behzad Zeinali, Dimitrios Karsinos, and Farshad Moradi. 2018. Progressive scaled STT-RAM for approximate computing in multimedia applications. *IEEE Trans. on Circuits and Systems II: Express Briefs* 65, 7 (2018), 938–942.

[292] Reza Zendegani, Mehdi Kamal, Milad Bahadori, Ali Afzali-Kusha, and Massoud Pedram. 2017. RoBA multiplier: A rounding-based approximate multiplier for high-speed yet energy-efficient digital signal processing. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 25, 2 (2017), 393–401.

[293] Reza Zendegani, Mehdi Kamal, Arash Fayyazi, Ali Afzali-Kusha, Saeed Safari, and Massoud Pedram. 2016. SEERAD: A high speed yet energy-efficient rounding-based approximate divider. In *Design, Automation and Test in Europe (DATE)*. 1481–1484.

[294] Georgios Zervakis, Fotios Ntouskas, Sotirios Xydis, Dimitrios Soudris, and Kiamal Pekmestzi. 2018. VOSsim: A framework for enabling fast voltage overscaling simulation for approximate computing circuits. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 26, 6 (2018), 1204–1208.

[295] Georgios Zervakis, Sotirios Xydis, Dimitrios Soudris, and Kiamal Pekmestzi. 2019. Multi-level approximate accelerator synthesis under voltage island constraints. *IEEE Trans. on Circuits and Systems II: Express Briefs* 66, 4 (2019), 607–611.

[296] Guowei Zhang and Daniel Sanchez. 2018. Leveraging hardware caches for memoization. *IEEE Computer Architecture Letters* 17, 1 (2018), 59–63.

[297] Jeff Zhang, Kartheek Rangineni, Zahra Ghodsi, and Siddharth Garg. 2018. ThUnderVolt: Enabling aggressive voltage underscaling and timing error resilience for energy efficient deep learning accelerators. In *Design Automation Conference (DAC)*. 1–6.

[298] Qian Zhang, Ting Wang, Ye Tian, Feng Yuan, and Qiang Xu. 2015. ApproxANN: An approximate computing framework for artificial neural network. In *Design, Automation and Test in Europe (DATE)*. 701–706.

[299] Xuhong Zhang, Jun Wang, and Jiangling Yin. 2016. Sapprox: Enabling efficient and accurate approximations on sub-datasets with distribution-aware online sampling. *Proceedings of the VLDB Endowment* 10, 3 (2016), 109–120.

[300] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. 2016. DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients. 1–13.

[301] Feiyu Zhu, Shaowei Zhen, Xilin Yi, Haoran Pei, Bowen Hou, and Yajuan He. 2022. Design of approximate radix-256 Booth encoding for error-tolerant computing. *IEEE Trans. on Circuits and Systems II: Express Briefs* 69, 4 (2022), 2286–2290.

Received 18 July 2023; revised 16 October 2024; accepted 24 December 2024